

Software Architecture

Architecture Methodology

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

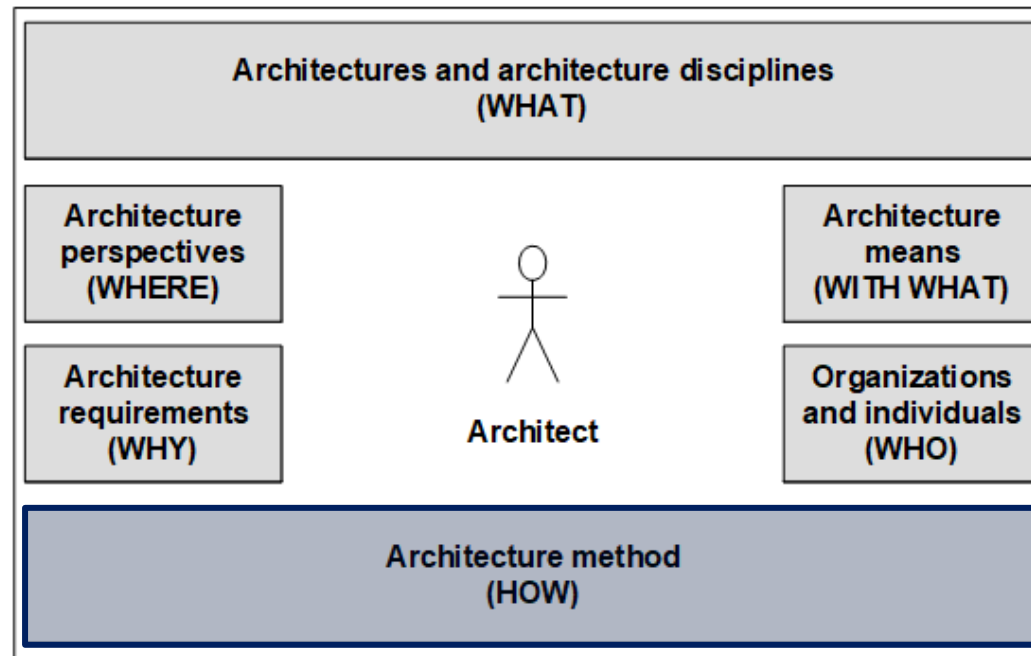
These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Lecture Opening

Architecture Orientation Framework



Architecture HOW discusses the process of systematically elaborating an architecture solution in response to a corresponding problem – in other words, with **architecture development models** [Vogel, Arnold et al 2011].



Lecture Opening

Motivation

By architecture we mean the **process of developing architecture as well as the result of this process.**

To the extent that architecture view models structure the manifestation of holistic architecture representation, **methodologies systematize the process of architecture planning, development and validation.**

Methodologies and view models go hand in hand, which is why the architecture frameworks referenced above also include process models in addition to view models.

Lecture Opening

Learning Objectives

You ...

- gain an overview of the basic features of modern architecture methodologies.
- understand that methodologies are adapted to the contexts in which they are used.
- know the role of architecture mandates for architecture methodologies.
- understand the central components as well as attitudes (i.e., evolutionary approach) of solution architecture methodologies.

Lecture Agenda



- Methodology
- Architecture Mandate
- Solution Architecture Methodology

Methodology

Overview

A **methodology systematically directs efforts and activities toward defined objectives**. It specifies the valid state and transition space for its respective field of concern.

A methodology includes a method in which it **outlines a systematic, step-by-step procedure**. Each step in the procedure specifies what information is needed to execute the step, what information is generated by the step, and what role is responsible for performing the step.

A methodology embodies the **deep structure of its method** and other **complementary building blocks** regardless of their instantiation.

Methodologies **facilitate discussions about the appropriateness of designs** while abstracting from their material implementation aspects.

Methodologies are **inevitably linked to method-specific view models** – thus practically inseparable from them.

Methodology

Overview

An architecture methodology increases the **systematicity and repeatability of architecture governance and performance activities** in an enterprise.

Methodologies **promote alignment, standardization, synergy, and economies of scale** – ultimately, methodologies promote an attitude of repetition rather than reinvention.

Architecture methodologies tangibly **explain the *why, what, who, when, and how* of states and state transitions** for their particular areas of responsibility.

Different types of architecture methodologies exist:

- **Solution Architecture Development** (e.g., Unified Process, TOGAF, ArchiMate,)
- **Enterprise and Domain Architecture Elaboration** (e.g., TOGAF, Zachman)
- **Reference Architecture Elaboration** (e.g., architecture pattern or roadmap methodologies)
- **Architecture Assessment Methodology** (e.g., ATAM).

Methodology

Overview

In the following slides, I will **outline** the **fundamental aspects** of **Architecture Methodologies** based on Open Group's **TOGAF®** framework [TOGAF 2021] and the **ADM** (Architecture Development Method) embedded within it.

The **view** I take **on architecture elaboration spans across** the **3 Architecture Disciplines** and **Levels** (Enterprise, Domain and Solution Architecture), while **it does not yet go down** to the next **level of detail embedded** in **Solution Architecture**, namely **Software Architecture**.

In the chapter on **Solution Architecture Methodology**, I will then **take a more detailed look** at **Solution Architecture**, **focusing** in particular on the level of **Software Architecture** itself.

However, **before we take a closer look** at the **TOGAF ADM method**, I want to **consider a few crucial but very general aspects** of **Architecture Frameworks** , first.

Methodology

Common Misconceptions

Before we **look** more closely at what **Architecture Frameworks** are, let's **clarify** what they are not, or rather, **what Misconceptions** about **Frameworks** are **widespread**.

The saying goes "A fool with a tool is still a fool" ...

... or **sometimes** even more correctly "A fool with a tool is an armed fool".



Methodology

Common Misconceptions

Before we **look** more closely at what **Architecture Frameworks** are, let's **clarify** what they are not, or rather, **what Misconceptions** about **Frameworks** are **widespread**.

Something that is **often forgotten** (and therefore **misconceptualized**): **Enterprise Architecture Frameworks** are **tools** – nothing more and nothing less!

Enterprise Architecture Frameworks are a **means** – not an end.

Furthermore, **Enterprise Architecture Frameworks** are **generic means** that **must first be adapted** to a specific context.

One of the **most common misconceptions** among Enterprise Architects is to **give frameworks the status of a purpose** ...

... and to **lose sight** of the fact that **they themselves** must **first recognise their purpose** in order to **adapt a generic framework** to an appropriate end.

The saying goes "**A fool with a tool is still a fool**" ...

... or **sometimes even more correctly** "**A fool with a tool is an armed fool**".



Methodology

Framework Adaption

Framework Adaptation refers to the **process** of **tailoring a framework**, such as TOGAF, to **fit the specific context** and **objectives** of a respective **situation**.

Banally speaking, all frameworks are a large number of written pages – in the end a pile of thick books.

Hence, to **derive maximum benefit** from **frameworks**, they **must be adapted** to a given **business context** and **objectives**.

The **art of applying frameworks** is to **distinguish the relevant** from the **irrelevant parts** of the **framework ...**

However, a **prerequisite** for this is that **it is clarified what** is to be **achieved** in the first place.



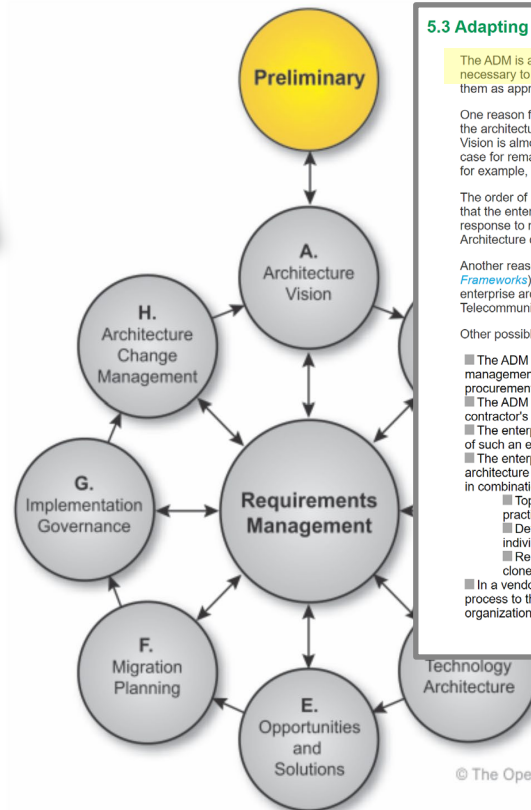
... to then **pick the really relevant cherries** and **use them**.



Methodology Framework Adaption

Framework Adaptation refers to the **process of tailoring a framework**, such as TOGAF, to **fit the specific context** and **objectives** of a respective **situation**.

Most frameworks explicitly point out the need for adaptation and guide through the respective steps as part of their process descriptions.



5.3 Adapting the ADM

The ADM is a generic method for architecture development, which is designed to deal with most system and organizational requirements. However, it will often be necessary to modify or extend the ADM to suit specific needs. One of the tasks before applying the ADM is to review its components for applicability, and then tailor them as appropriate to the circumstances of the individual enterprise. This activity may well produce an "enterprise-specific" ADM.

One reason for wanting to adapt the ADM, which it is important to stress, is that the order of the phases in the ADM is to some extent dependent on the maturity of the architecture discipline within the enterprise. For example, if the business case for doing architecture at all is not well recognized, then creating an Architecture Vision is almost always essential; and a detailed Business Architecture often needs to come next, in order to underpin the Architecture Vision, detail the business case for remaining architecture work, and secure the active participation of key stakeholders in that work. In other cases a slightly different order may be preferred; for example, a detailed inventory of the baseline environment may be done before undertaking the Business Architecture.

The order of phases may also be defined by the architecture principles and business principles of an enterprise. For example, the business principles may dictate that the enterprise be prepared to adjust its business processes to meet the needs of a packaged solution, so that it can be implemented quickly to enable fast response to market changes. In such a case, the Business Architecture (or at least the completion of it) may well follow completion of the Information Systems Architecture or the Technology Architecture.

Another reason for wanting to adapt the ADM is if TOGAF is to be integrated with another enterprise framework (as explained in [Part I, 2.10 Using TOGAF with Other Frameworks](#)). For example, an enterprise may wish to use TOGAF and its generic ADM in conjunction with the well-known Zachman Framework, or another enterprise architecture framework that has a defined set of deliverables specific to a particular vertical sector: Government, Defense, e-Business, Telecommunications, etc. The ADM has been specifically designed with this potential integration in mind.

Other possible reasons for wanting to adapt the ADM include:

- The ADM is one of the many corporate processes that make up the corporate governance model. It is complementary to, and supportive of, other standard program management processes, such as those for authorization, risk management, business planning and budgeting, development planning, systems development, and procurement.
- The ADM is being mandated for use by a prime or lead contractor in an outsourcing situation, and needs to be tailored to achieve a suitable compromise between the contractor's existing practices and the contracting enterprise's requirements.
- The enterprise is a small-to-medium enterprise, and wishes to use a "cut-down" method more attuned to the reduced level of resources and system complexity typical of such an environment.
- The enterprise is very large and complex, comprising many separate but interlinked "enterprises" within an overall collaborative business framework, and the architecture method needs to be adapted to recognize this. Different approaches to planning and integration may be used in such cases, including the following (possibly in combination):
 - Top-down planning and development - designing the whole interconnected meta-enterprise as a single entity (an exercise that typically stretches the limits of practicality)
 - Development of a "generic" or "reference" architecture, typical of the enterprises within the organization, but not representing any specific enterprise, which individual enterprises are then expected to adapt in order to produce an architecture "instance" suited to the particular enterprise concerned.
 - Replication - developing a specific architecture for one enterprise, implementing it as a proof-of-concept, and then taking that as a "reference architecture" to be cloned in other enterprises.
- In a vendor or production environment, a generic architecture for a family of related products is often referred to as a "Product Line Architecture", and the analogous process to that outlined above is termed "(Architecture-based) Product Line Engineering". The ADM is targeted primarily at architects in IT user enterprises, but a vendor organization whose products are IT-based might well wish to adapt it as a generic method for a Product Line Architecture development.

© The Open Group

Methodology

Framework Adaption (Objectives)

You **need to know** where you **want to go before** you can make a sensible **decision** about the **path** you choose **to take**. For the meaningful adaptation of an Architecture Framework, you need to know what your objectives are.

Now, how do you clarify what is to be achieved in the first place?

You have to answer the question of what the specific benefit of Architecture is for your Enterprise.

In other words: what does your Organization expect from Architecture?

Clarifying the question of the Architecture value proposition is itself a process and virtually never ends.

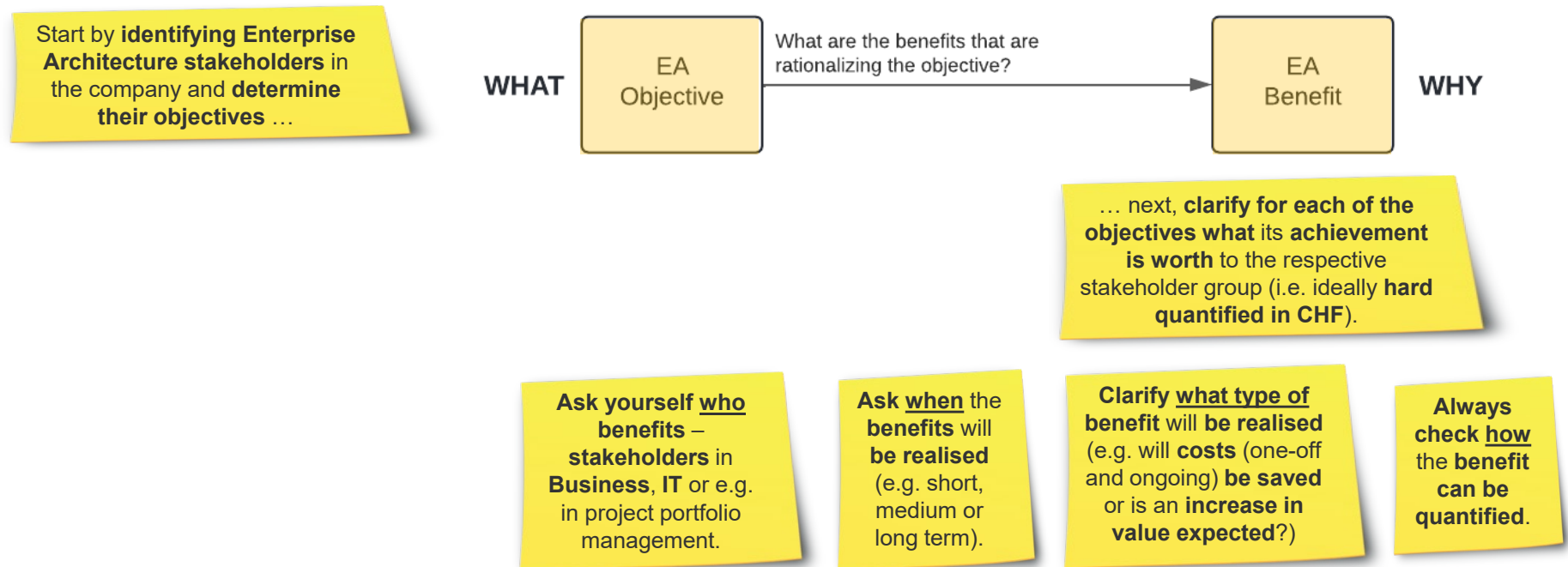
However, frameworks can only be adapted sensibly once this question has been clarified (at least for an initial moment).



Methodology

Framework Adaption (Objectives)

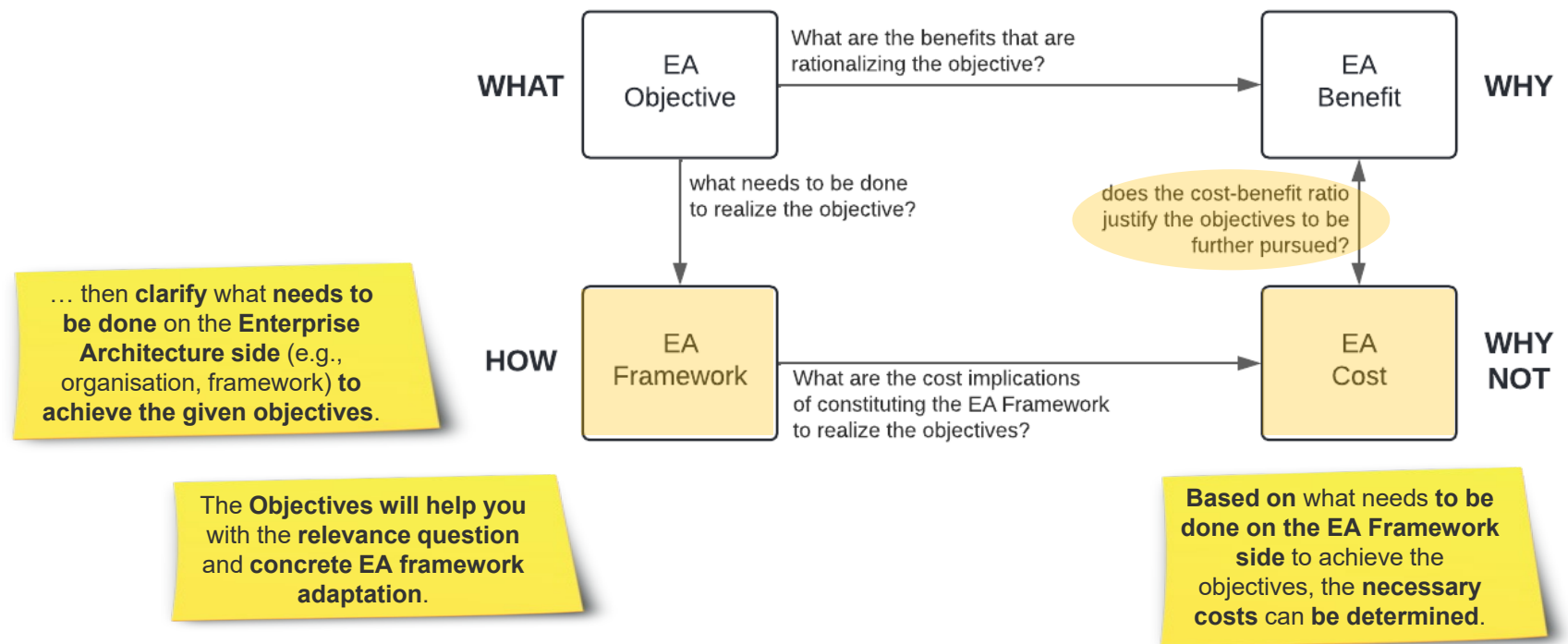
You **need to know** where you **want to go before** you can make a sensible **decision** about the **path** you choose **to take**. For the meaningful adaptation of an Architecture Framework, you need to know what your objectives are.



Methodology

Framework Adaption (Objectives)

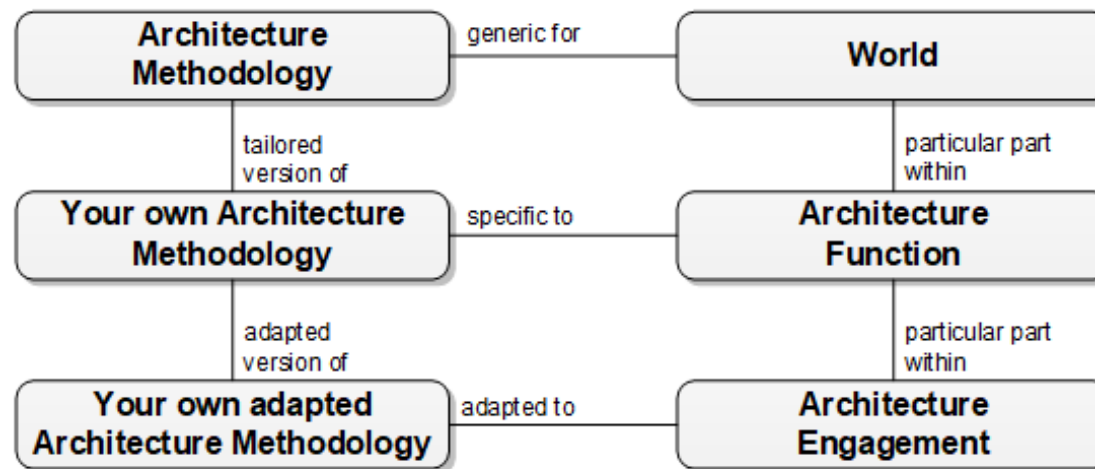
You **need to know** where you **want to go before** you can make a sensible **decision** about the **path** you choose **to take**. For the meaningful adaptation of an Architecture Framework, you need to know what your objectives are.



Methodology

Framework Adaption

Finally, **Architecture** or **Enterprise Architecture Frameworks** and **Methodologies** usually need to be **adapted** at **different levels** to the **context** in which they are applied.

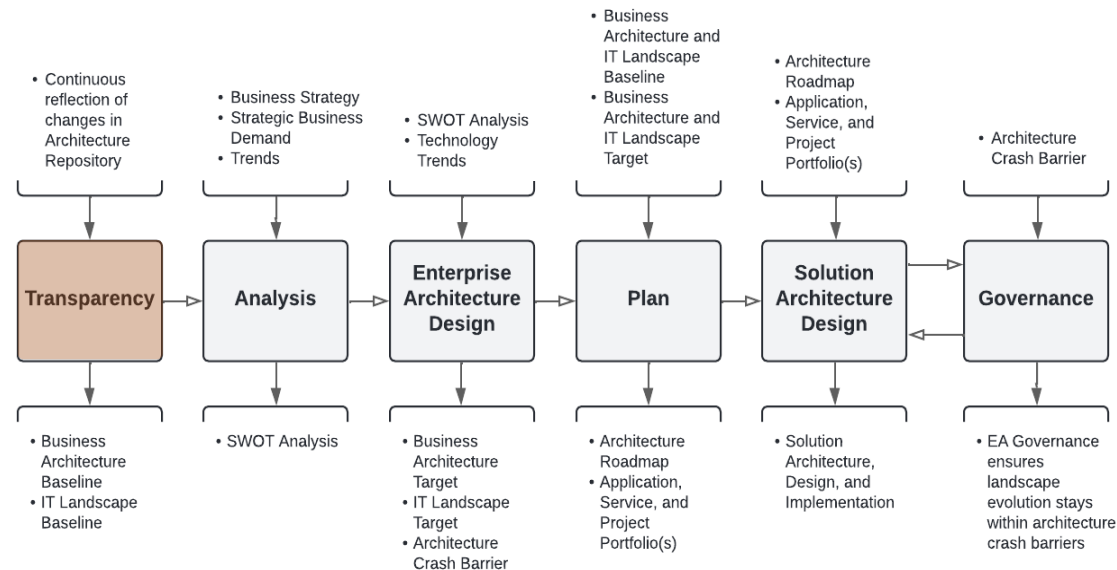


Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

Transparency regarding the **Business Architecture** and the supporting **IT landscape** is achieved through **continuous maintenance** of the As-Is situation (**Baseline**).



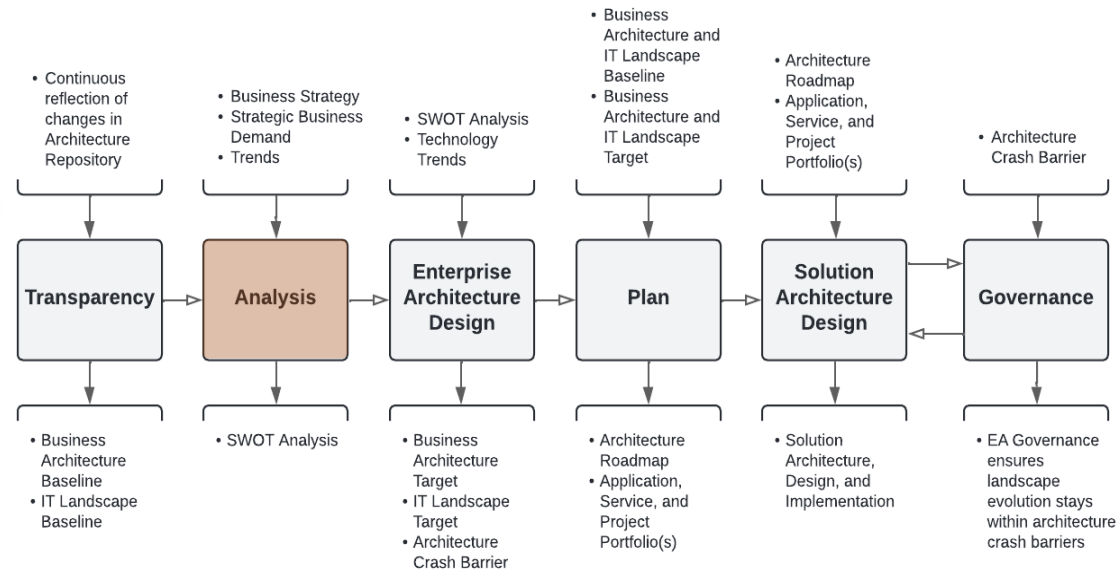
Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

Analysis focuses on inputs at a strategic level, such as the business strategy or strategic business demand.

In the lights of the **As-Is Situation**, the **Analysis** identifies systematic **deficits** and **gaps**, **redundancies**, **risks** but also **opportunities** that arise, for example, from technology trends.



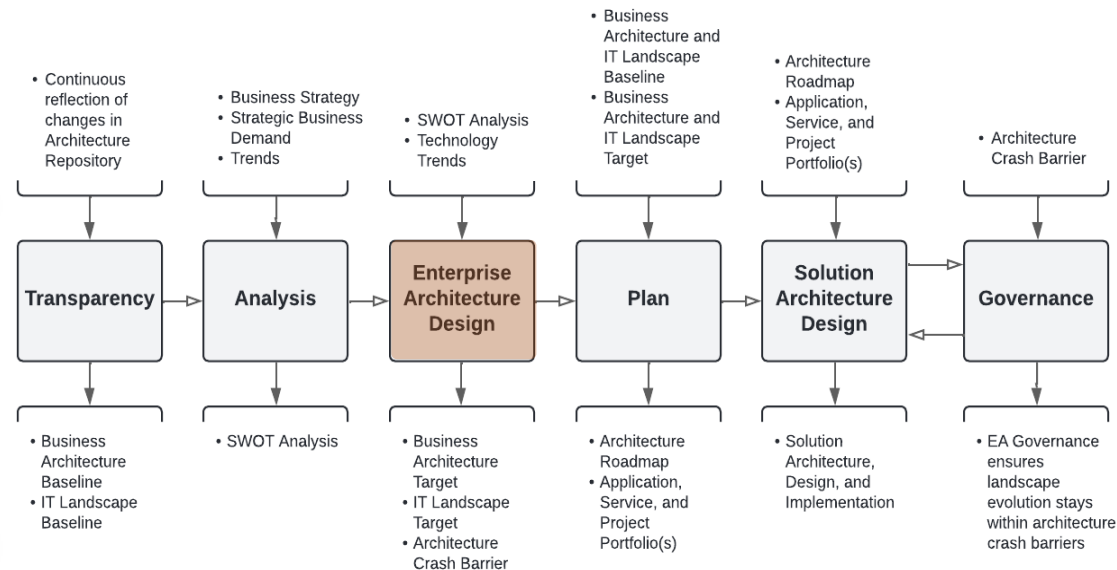
Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

During **EA Design**, the Enterprise Architect **develops** the **Business Architecture** (e.g. business capabilities, processes, organisation, products, information).

Further, the Enterprise Architect **derives** the **design** of the **target IT Landscape** (e.g. IT systems (application and data), technologies, platforms and infrastructure) **from** the **Business Architecture**.

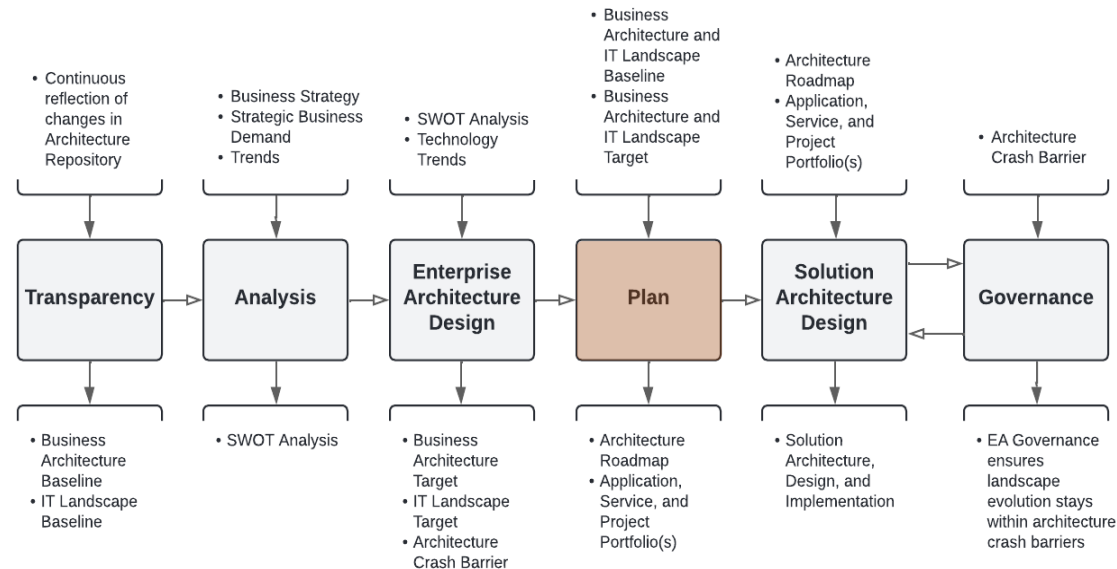


Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

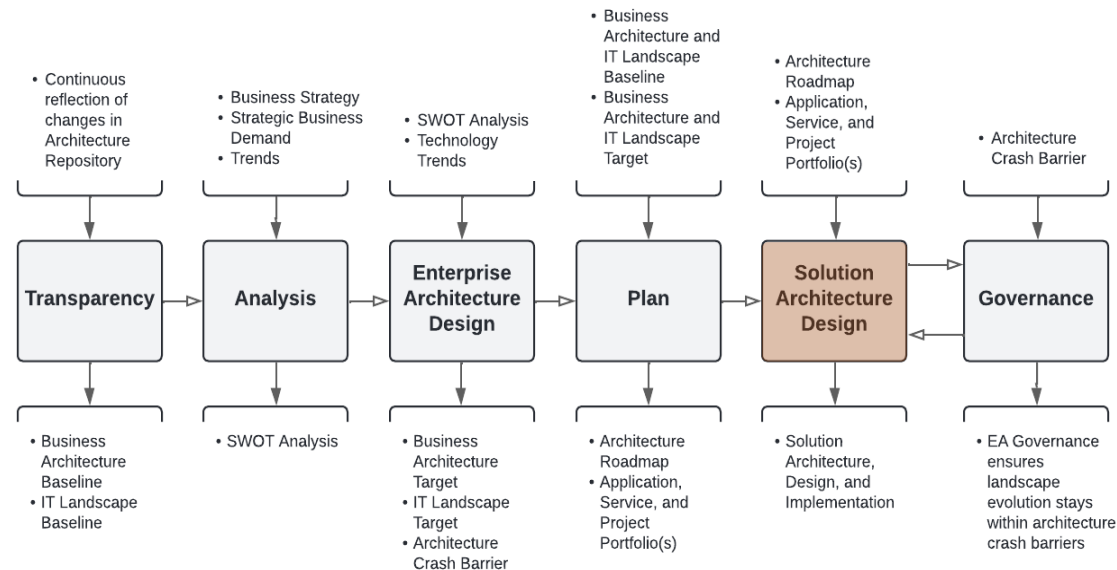
Plan comprises the planning of the **achievement** of the **Target Architecture**, which includes an **Architecture Roadmap** and **decision-making in portfolio management** (e.g. project, application or service portfolio).



Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



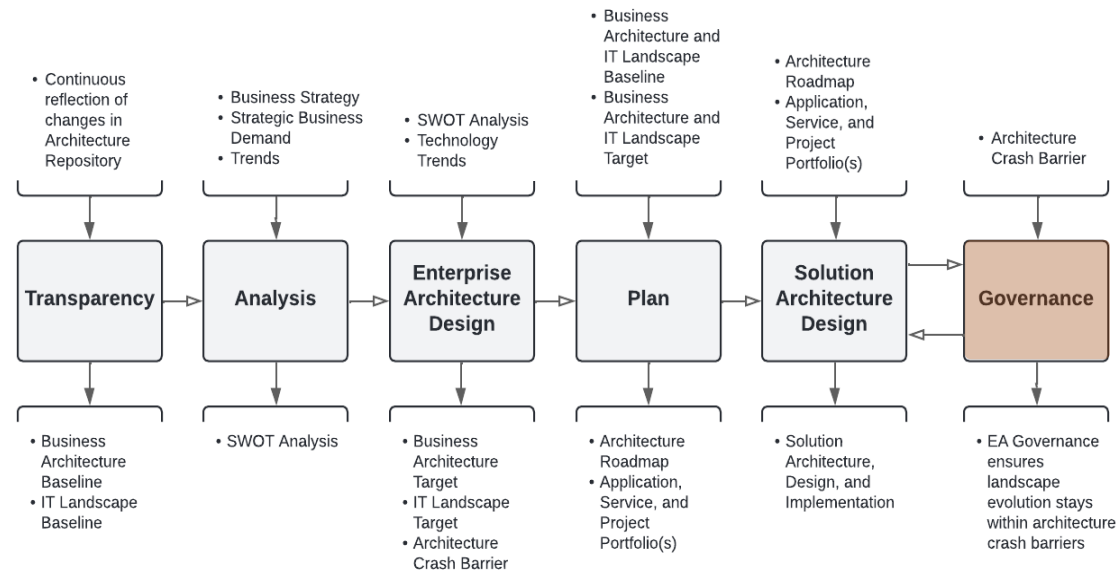
While **Enterprise Architecture** contributes the **landscape design**, **Solution Architects** deliver the **design** of individual **landscape building blocks**.

Remember that we noted: **Software Architecture** is **embedded / part of** the **Solution Architecture** discipline.

Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Governance, finally includes the definition of **guard rails** (for projects and maintenance tasks).

Guard rails are **criteria** that include **business, organisational, regulatory, principle or standard-related conditions** and have been significantly **shaped by Enterprise Architects**.

Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture of an organisation**.

Remember that we **already** looked this **end-2-end Lifecycle Management Schema** when we gained an overview of essential concepts (WHAT deck).



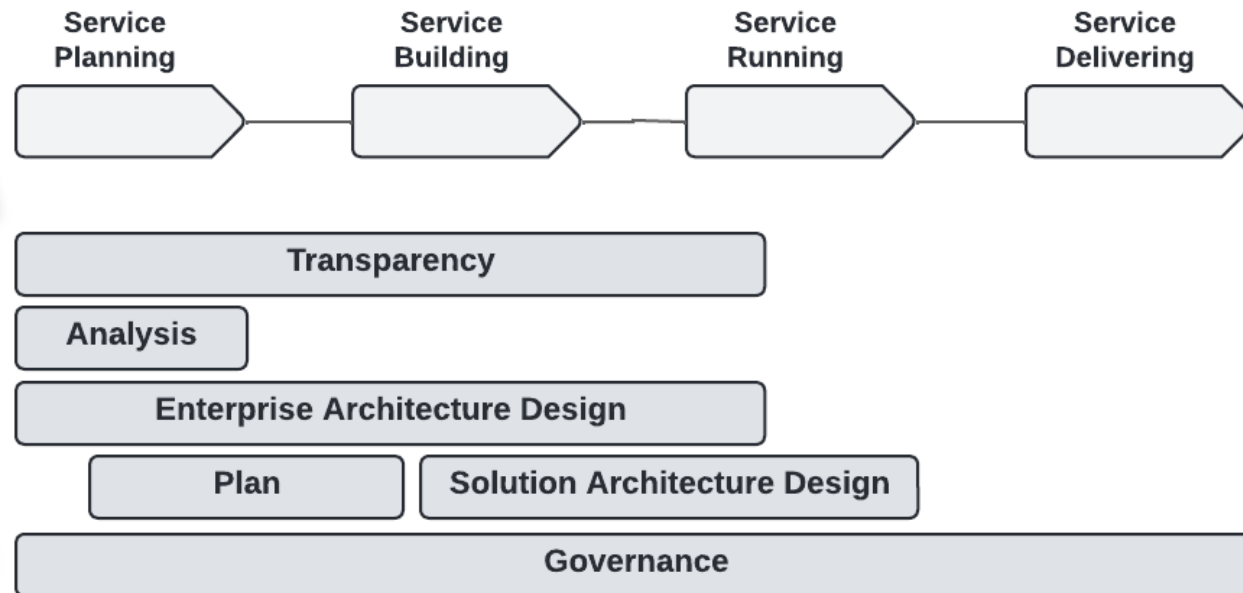
Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

Remember that we **already** looked this **end-2-end Lifecycle Management Schema** when we gained an overview of essential concepts (WHAT deck).

Here you see a **possible mapping** of the high-level **architecture process** (previous slide) to the **end-2-end lifecycle management perspective** within an enterprise.



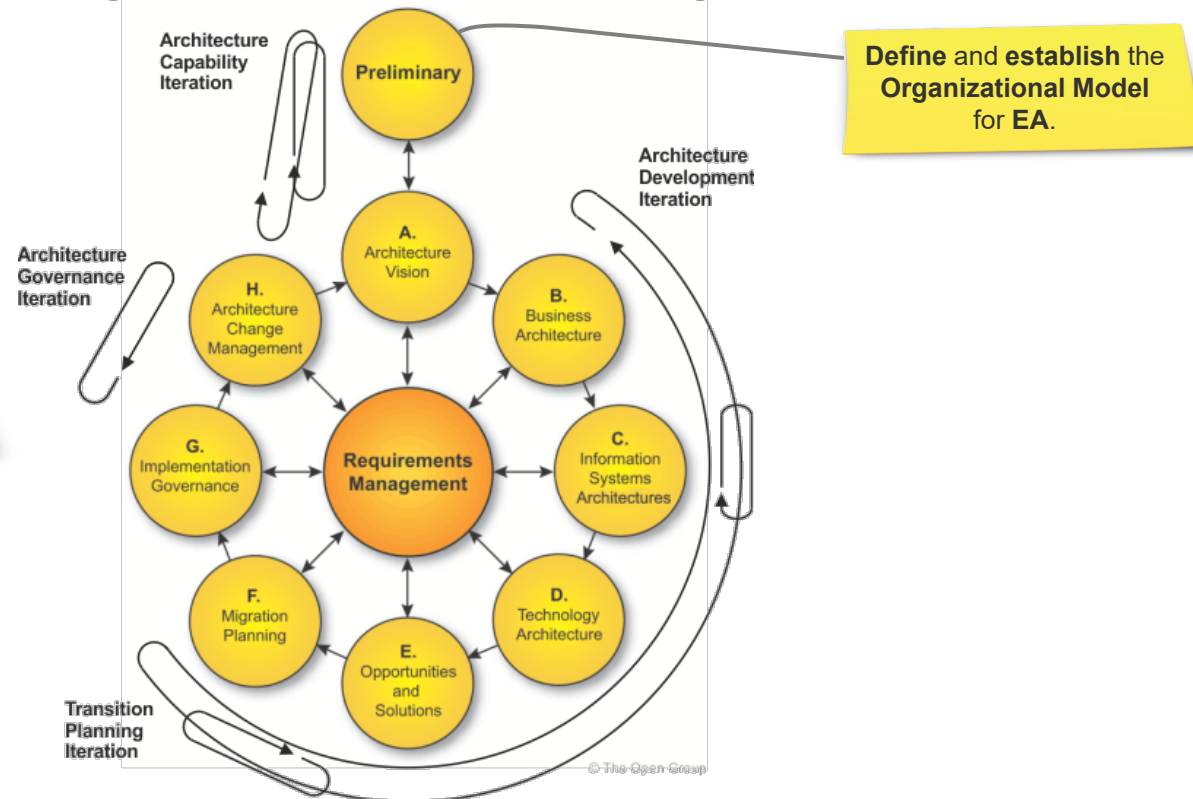
Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing and governing** the **architecture of an organisation**.

Using the **TOGAF ADM** as an **example**, we **acknowledge** **key aspects** of **architecture processes**.

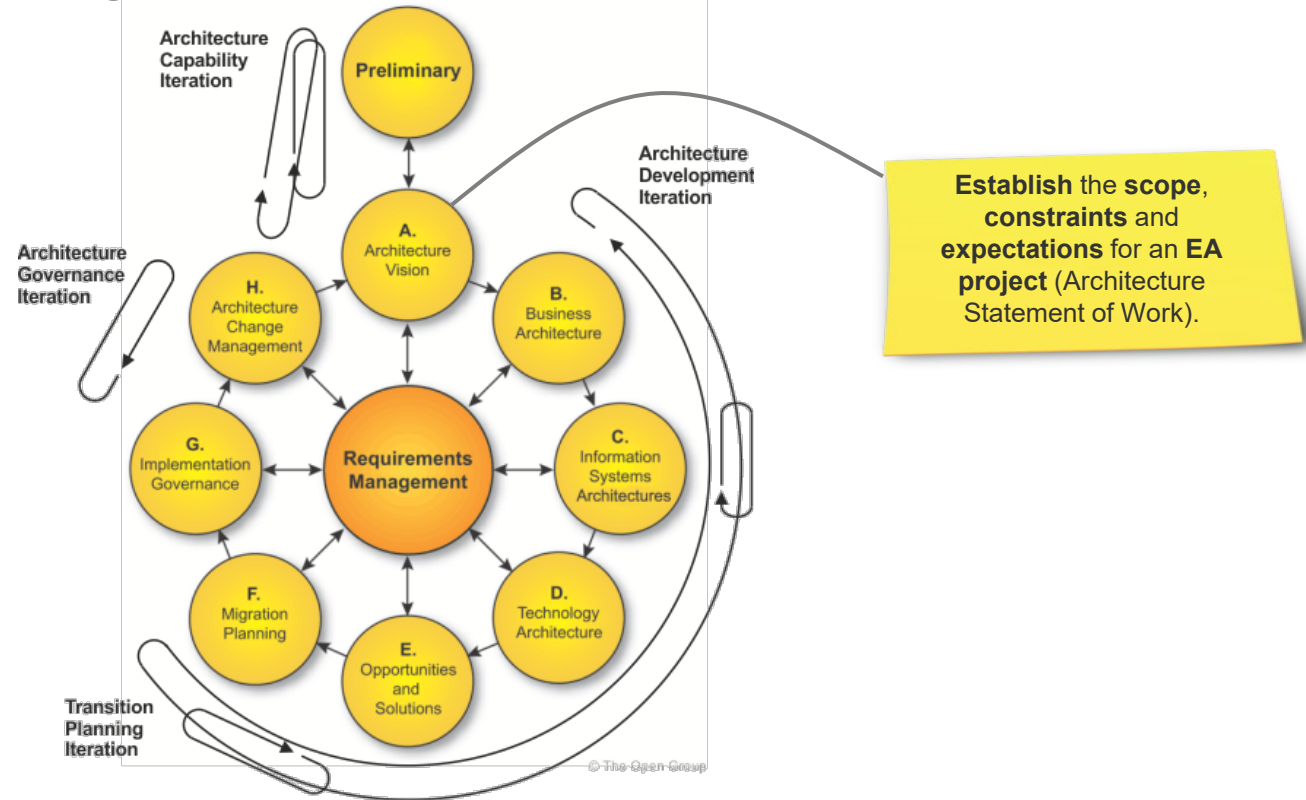
Architecture processes **generally** **comprise** **steps** for **planning, design, implementation, deployment and governance**.



Methodology

Architecture Process

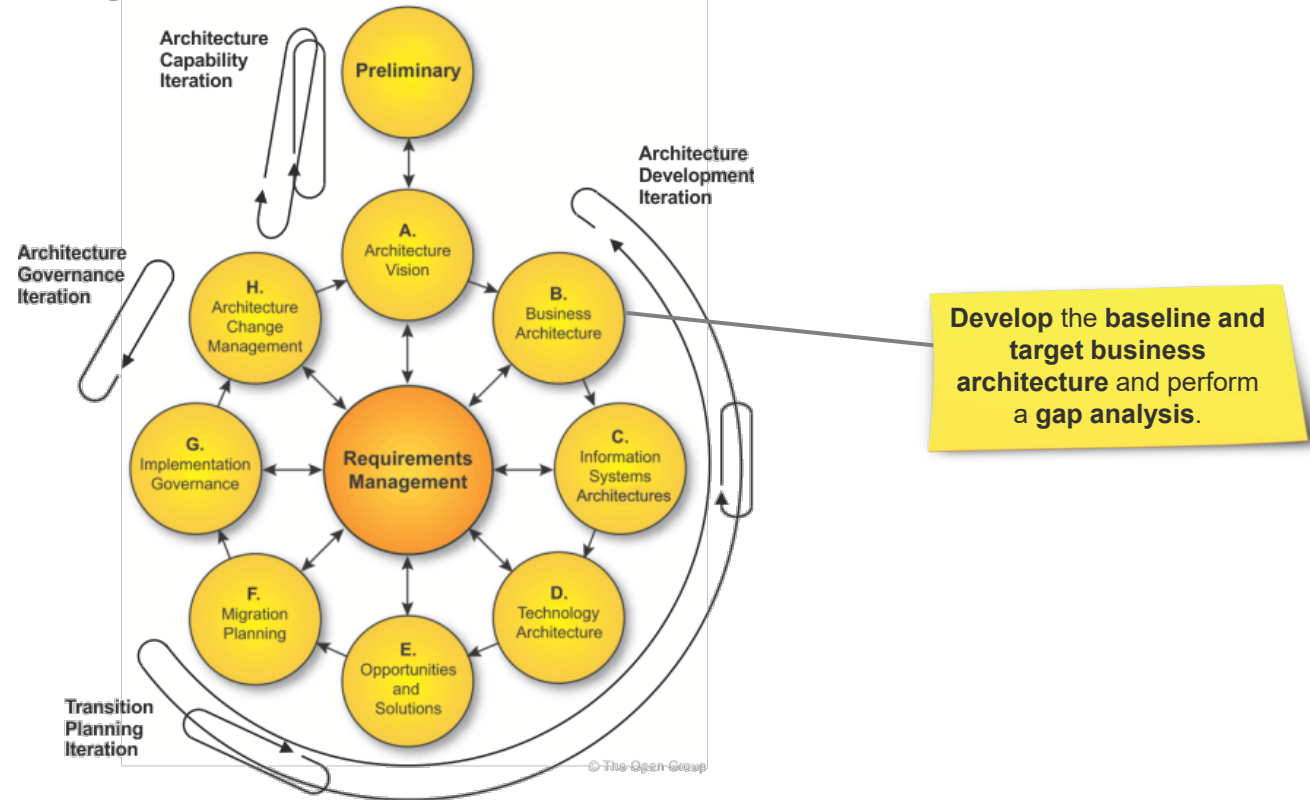
An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

Architecture Process

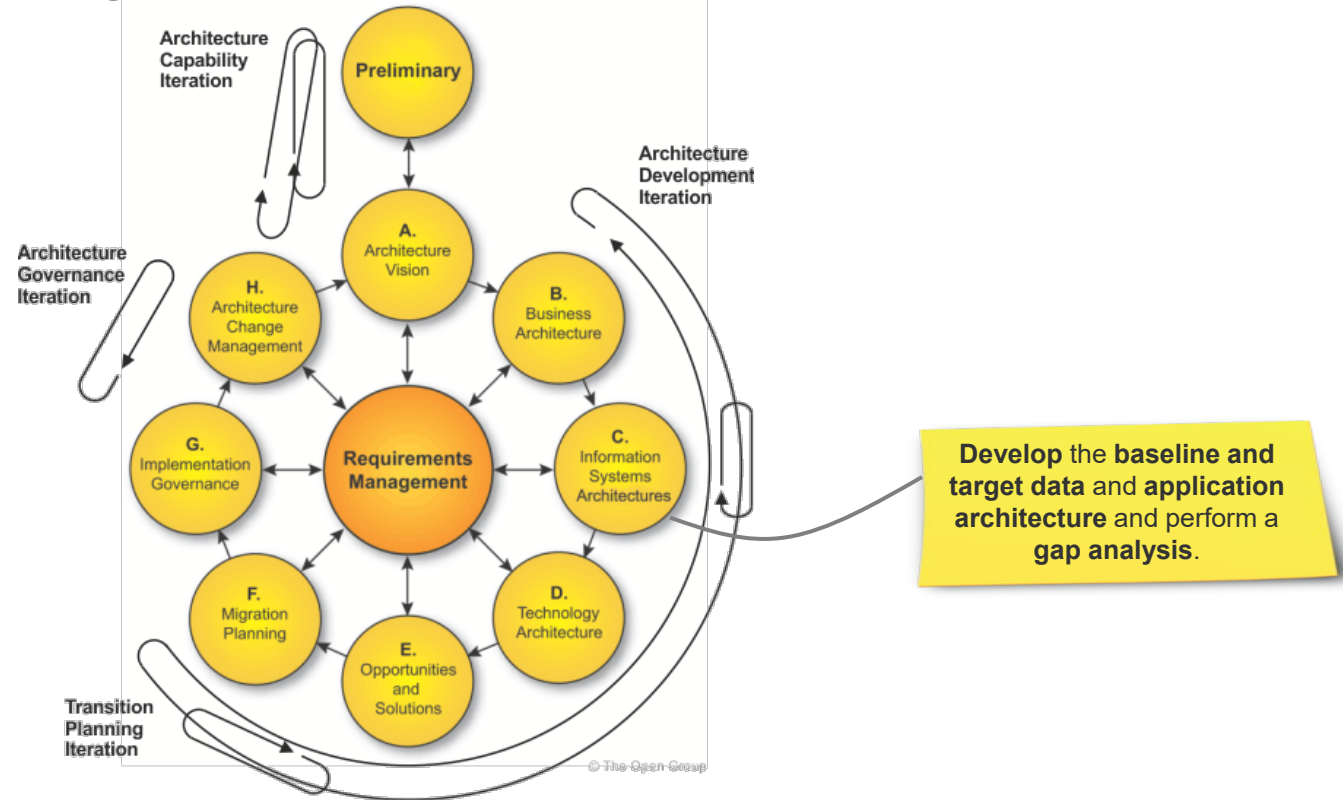
An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

Architecture Process

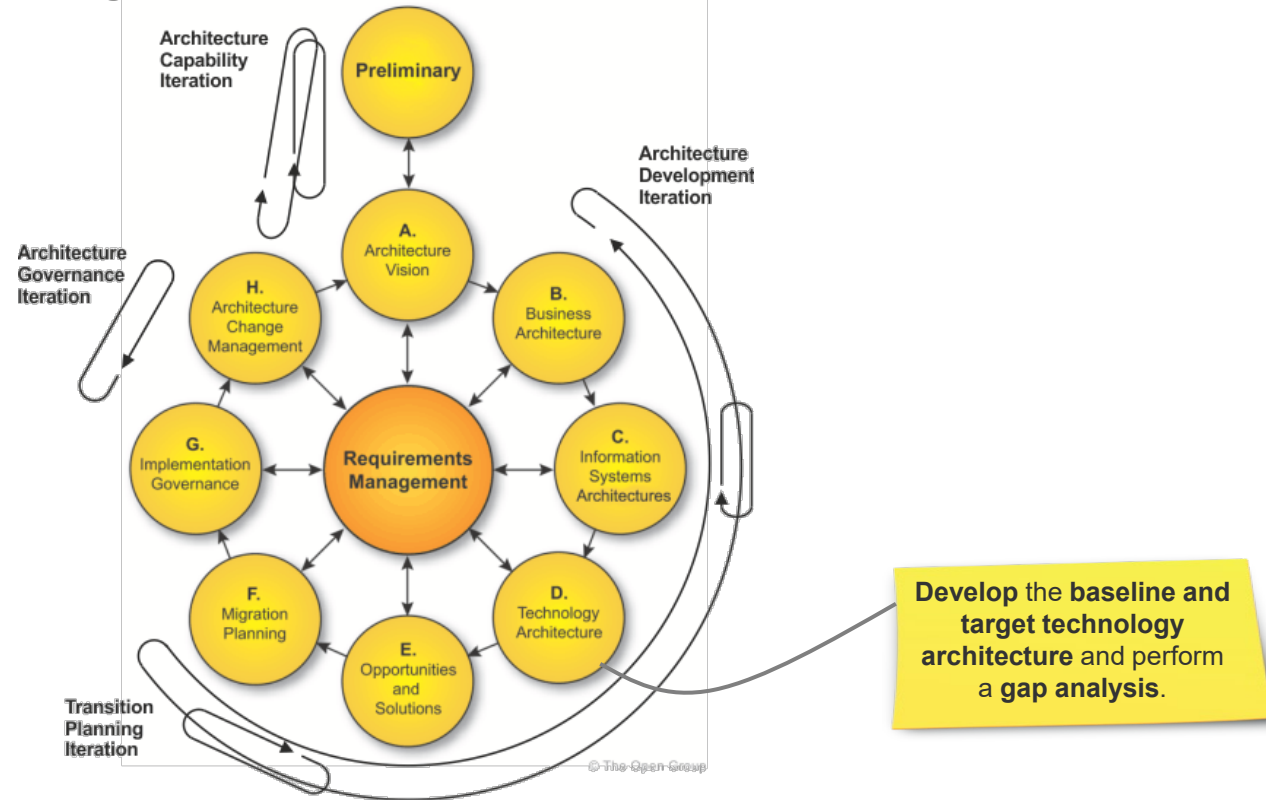
An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

Architecture Process

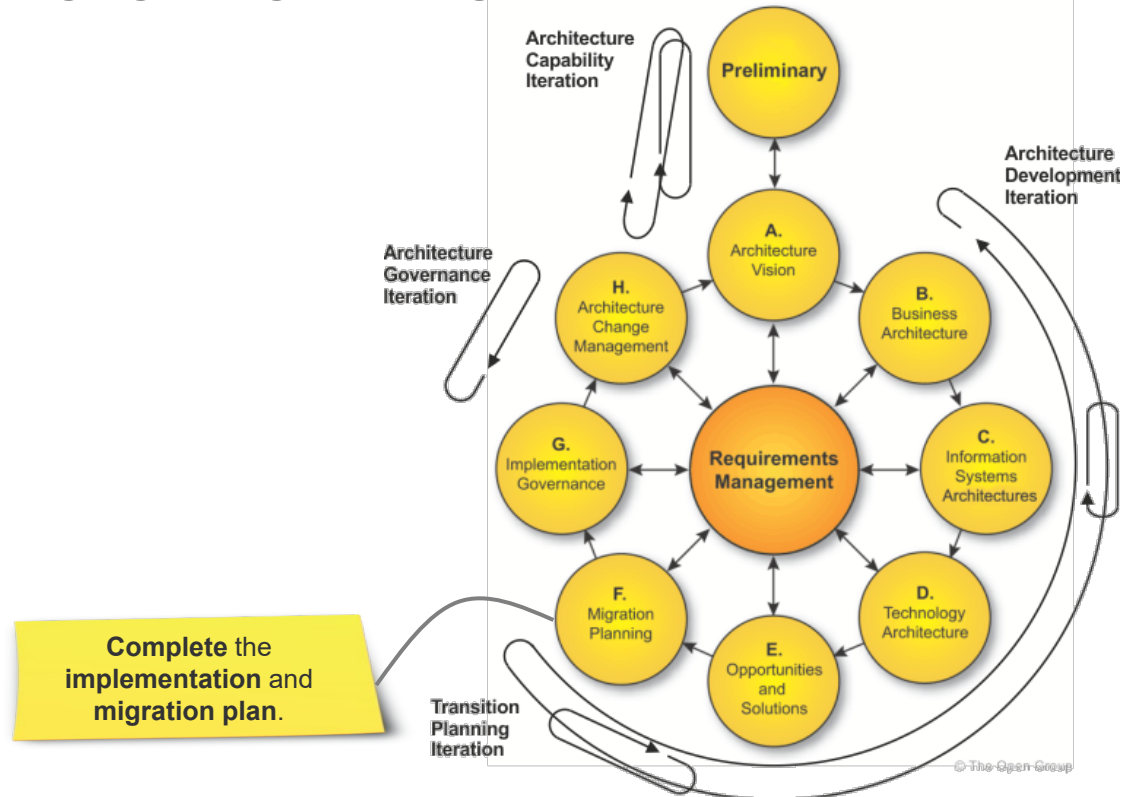
An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

Architecture Process

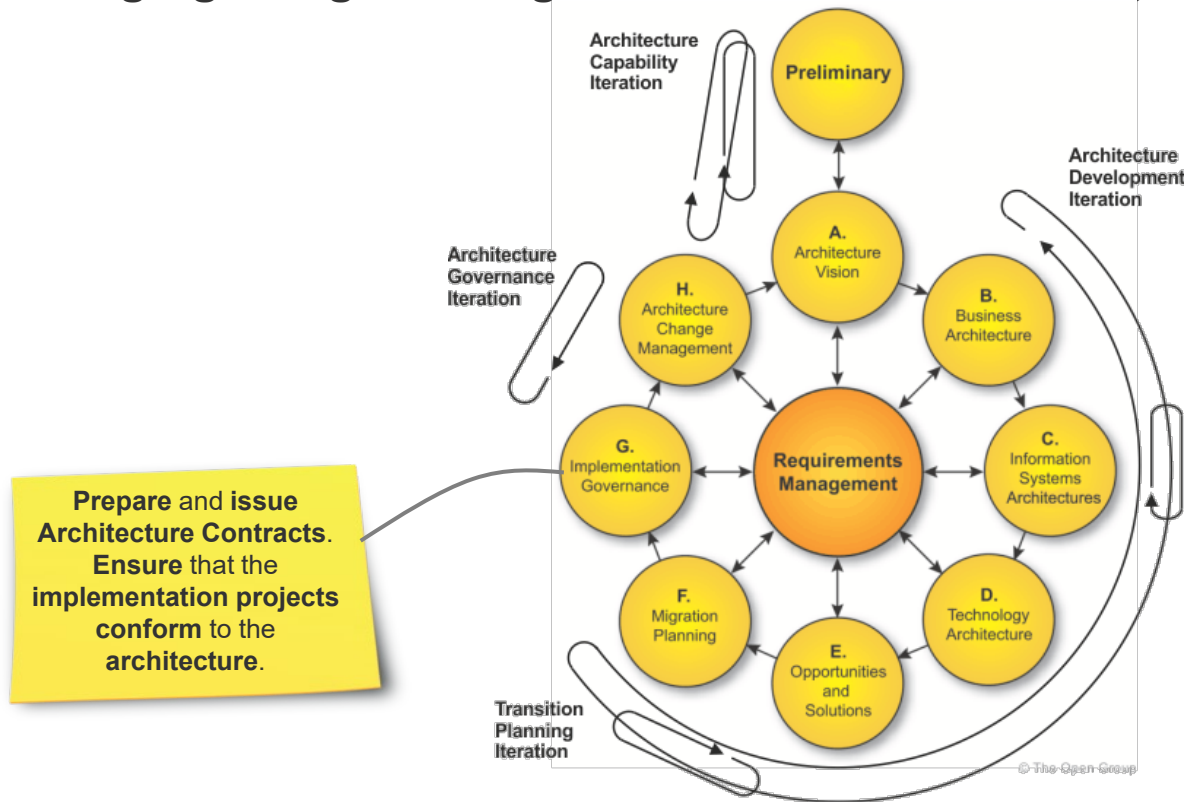
An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

Architecture Process

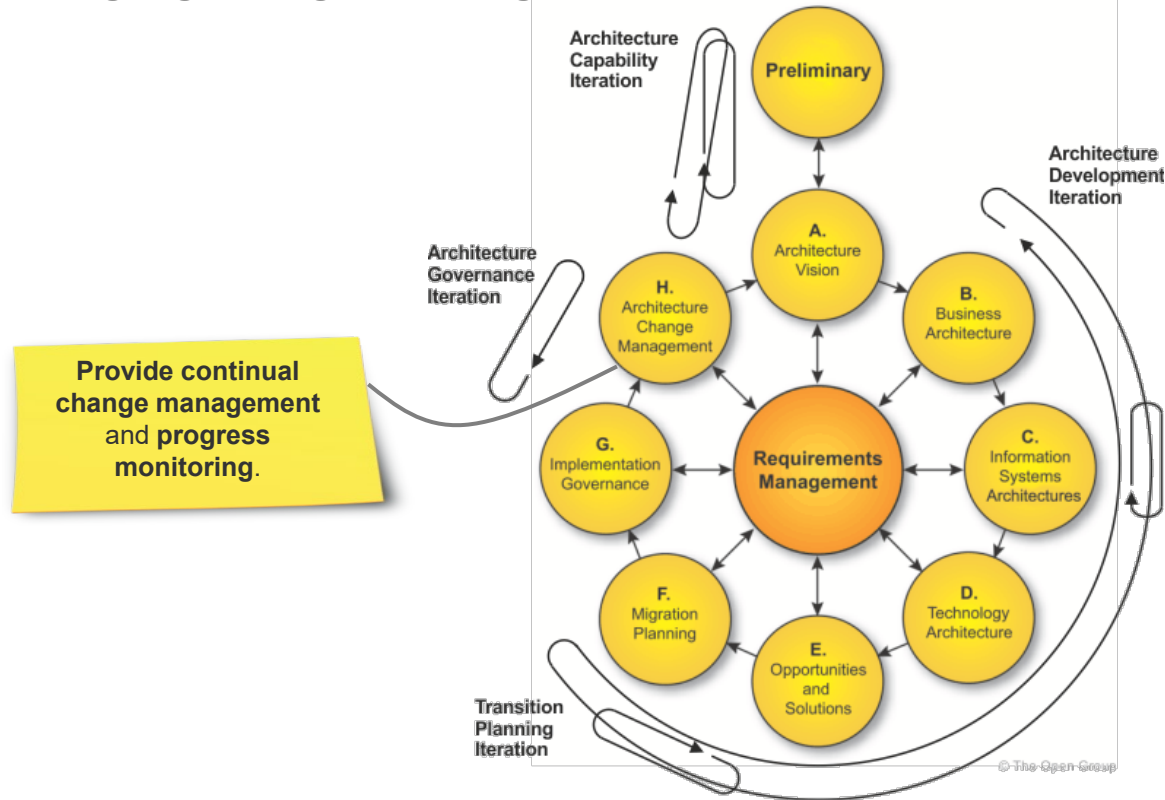
An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

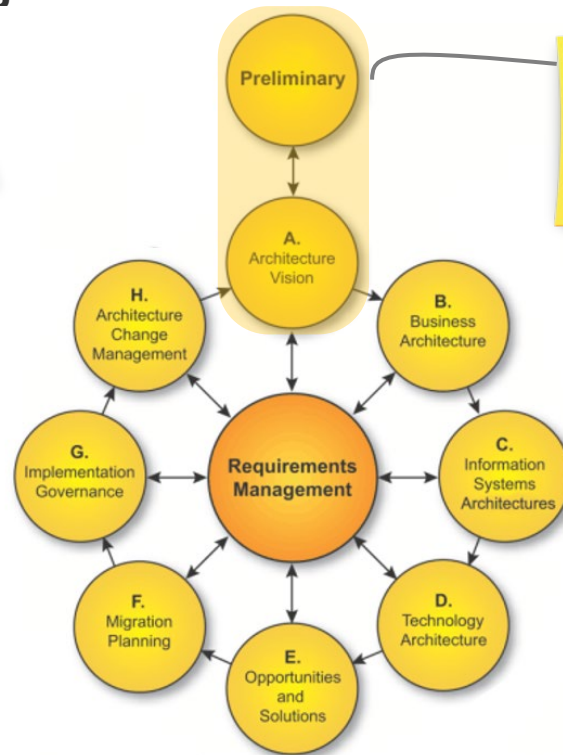
Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

Architecture processes also **favour** an **iterative, evolutionary** approach to architecture elaboration.

For example, **TOGAF ADM** distinguishes four major **iteration cycles**.

The **Architecture Context Cycle** is all about **setting up** your architecture practice and **launching** your EA project(s).

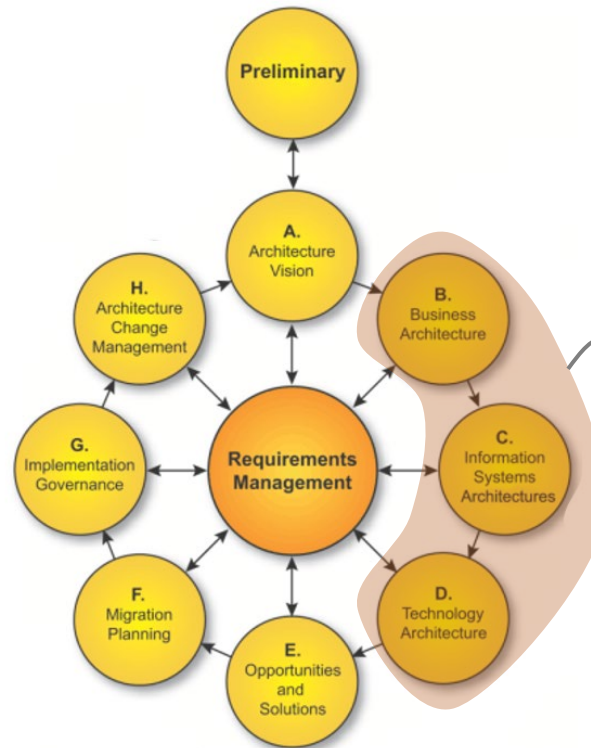


© The Open Group

Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



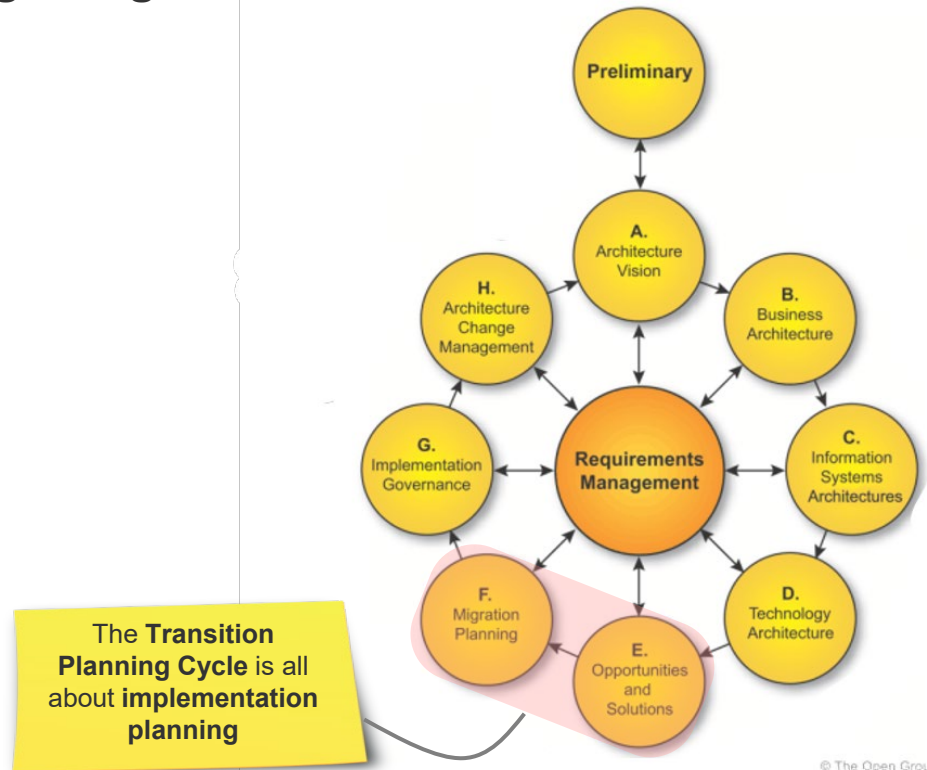
The **Architecture Delivery Cycle** is about **capturing** the **baseline architecture** (as-is), **fit-gap analysing** this, and finally **designing** the **target architecture** (to-be) from the perspectives of business, applications & data and platforms respectively.

© The Open Group

Methodology

Architecture Process

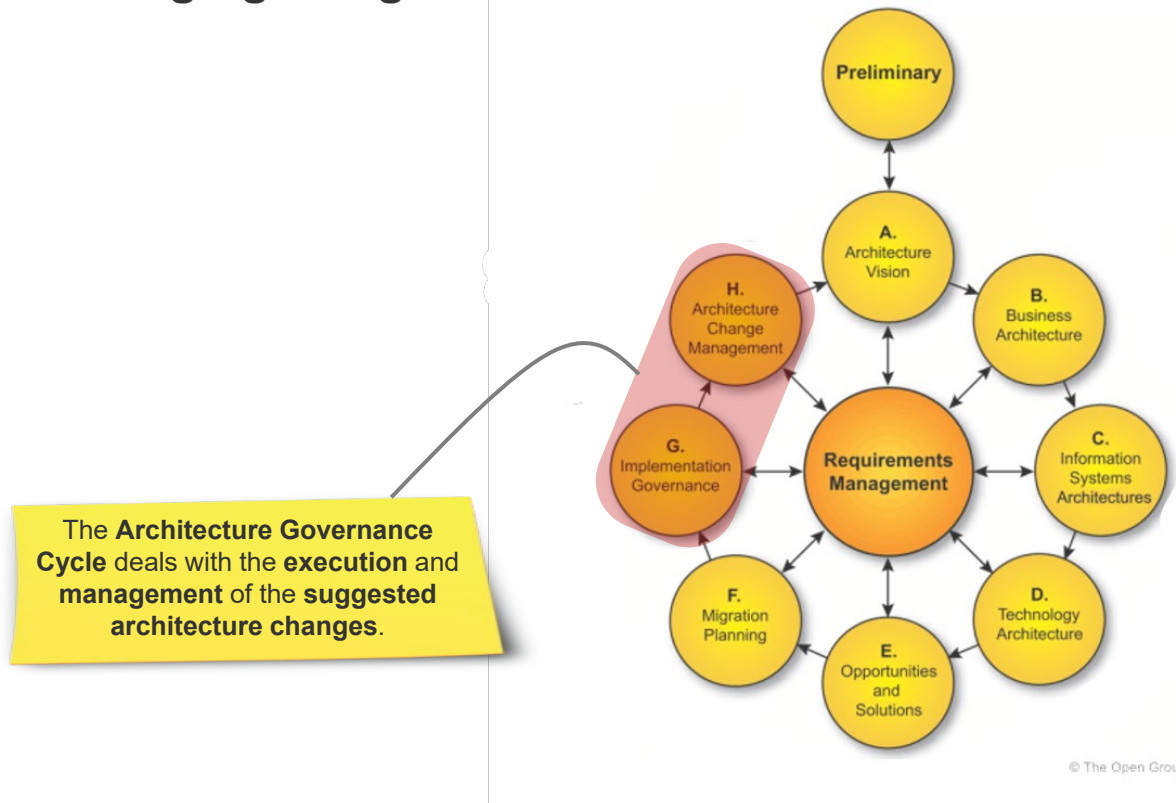
An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

Architecture Process

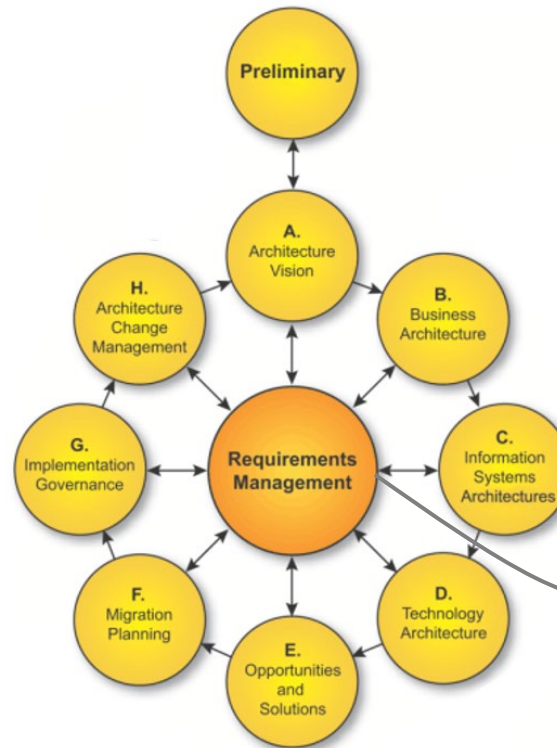
An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.



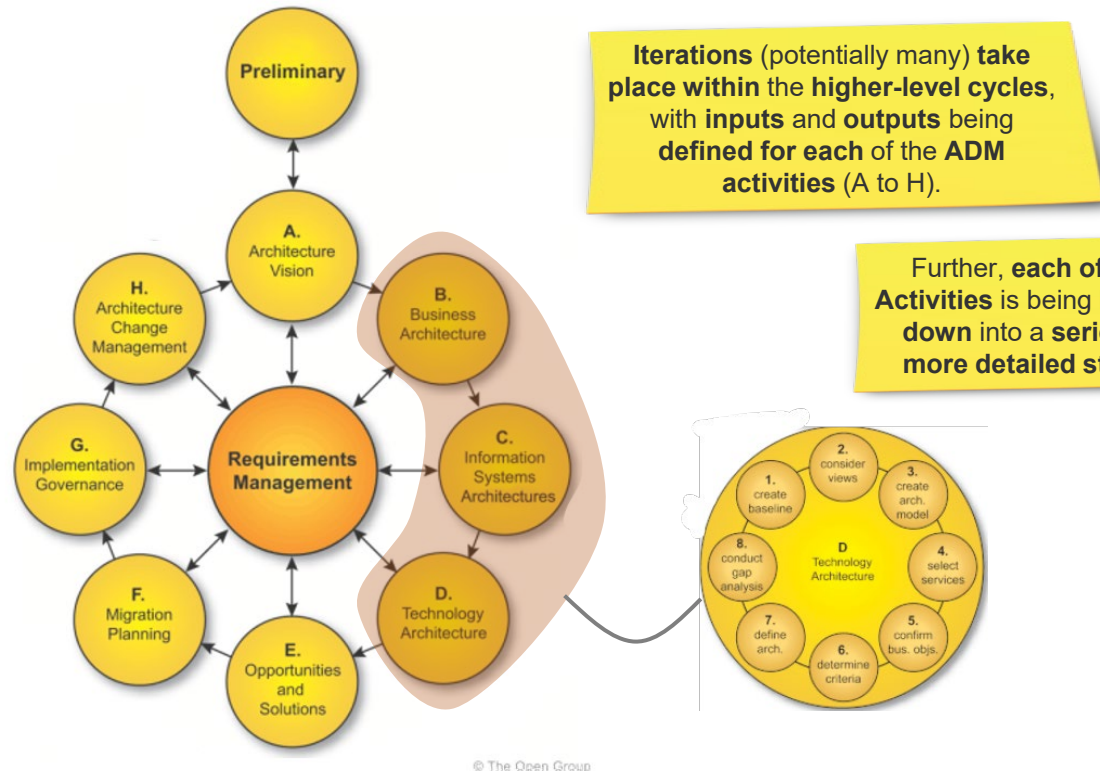
Requirements Management is not a cyclical phase but at the centre of the entire methodology where any decisions taken at during any phase of the initiative can be measured against its requirements.

© The Open Group

Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

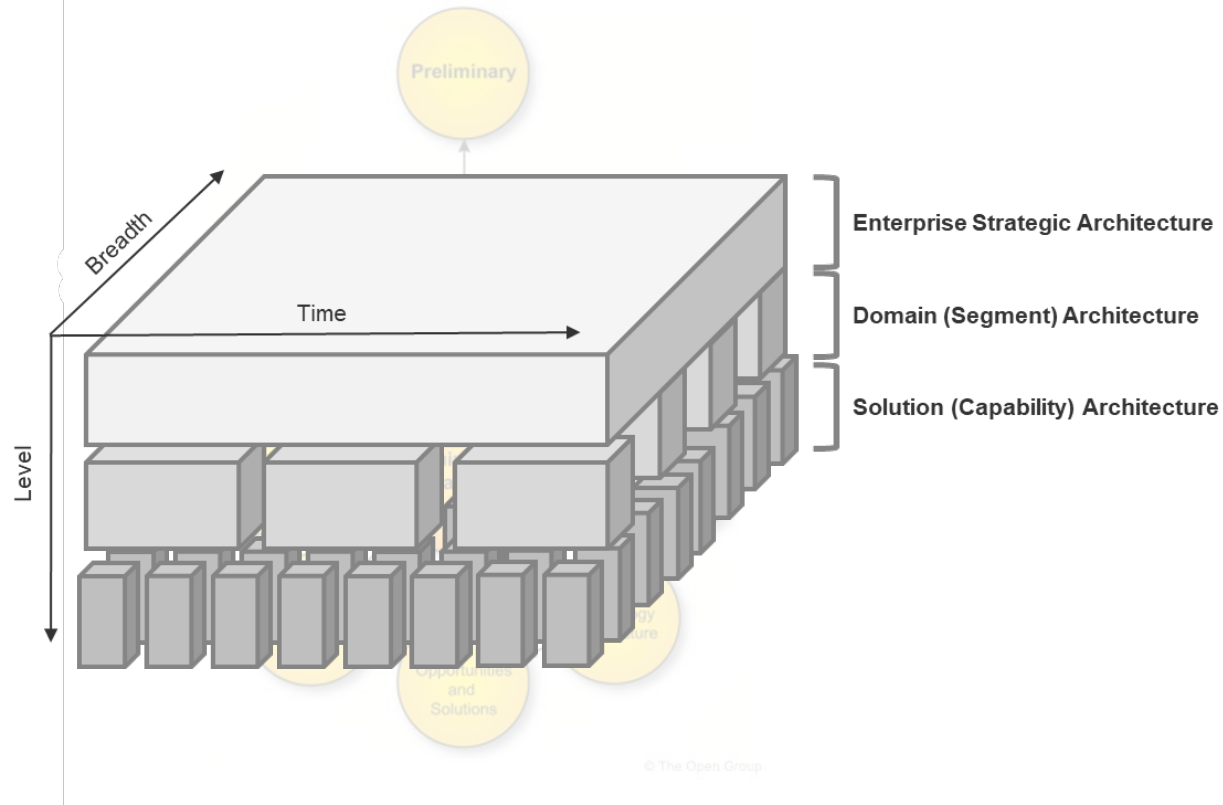


Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

Architecture Processes often consider different levels of granularity (Level), time horizons (Time) and breadth of perspectives (Breadth).

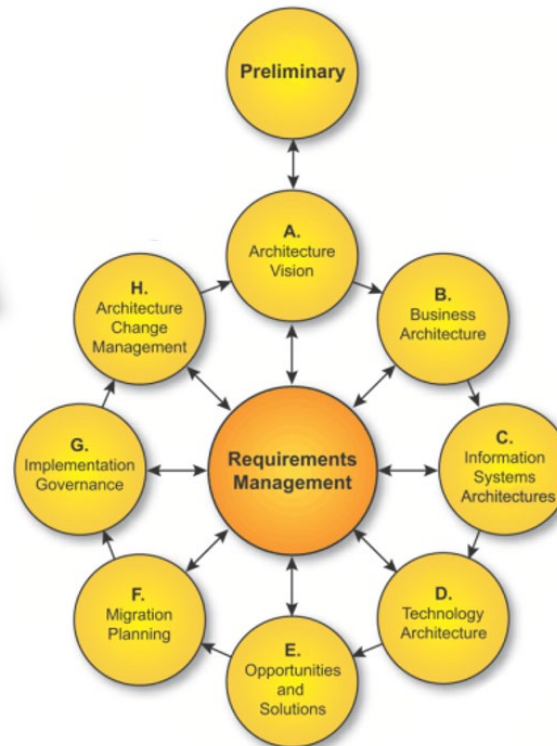


Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

The **TOGAF ADM** is applied on each of the **three levels** (Enterprise, Domain and Solution Architecture), whereby the **process-related interlocking** works as illustrated on the following slides.



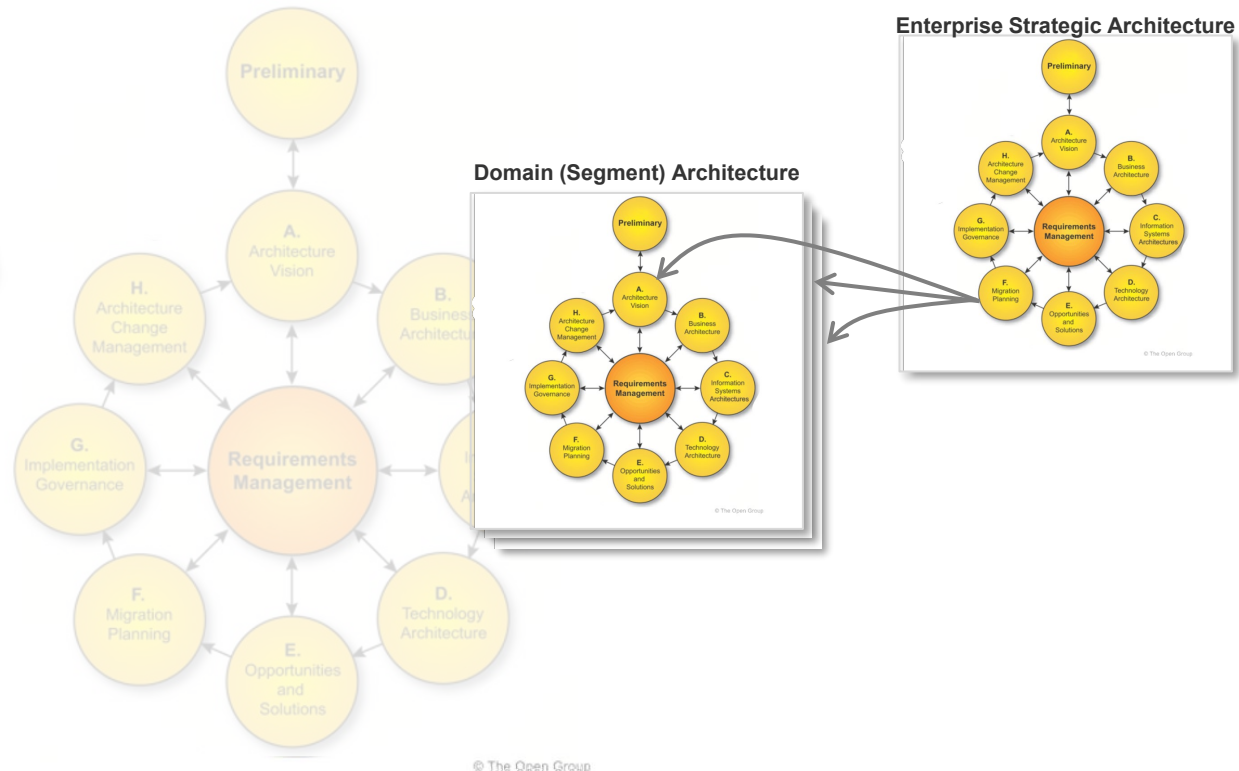
© The Open Group

Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

While the **solution perspective** at **enterprise level** is a **strategic and overarching one**, from which **initiatives at domain-level** (aka: segment) are derived ...



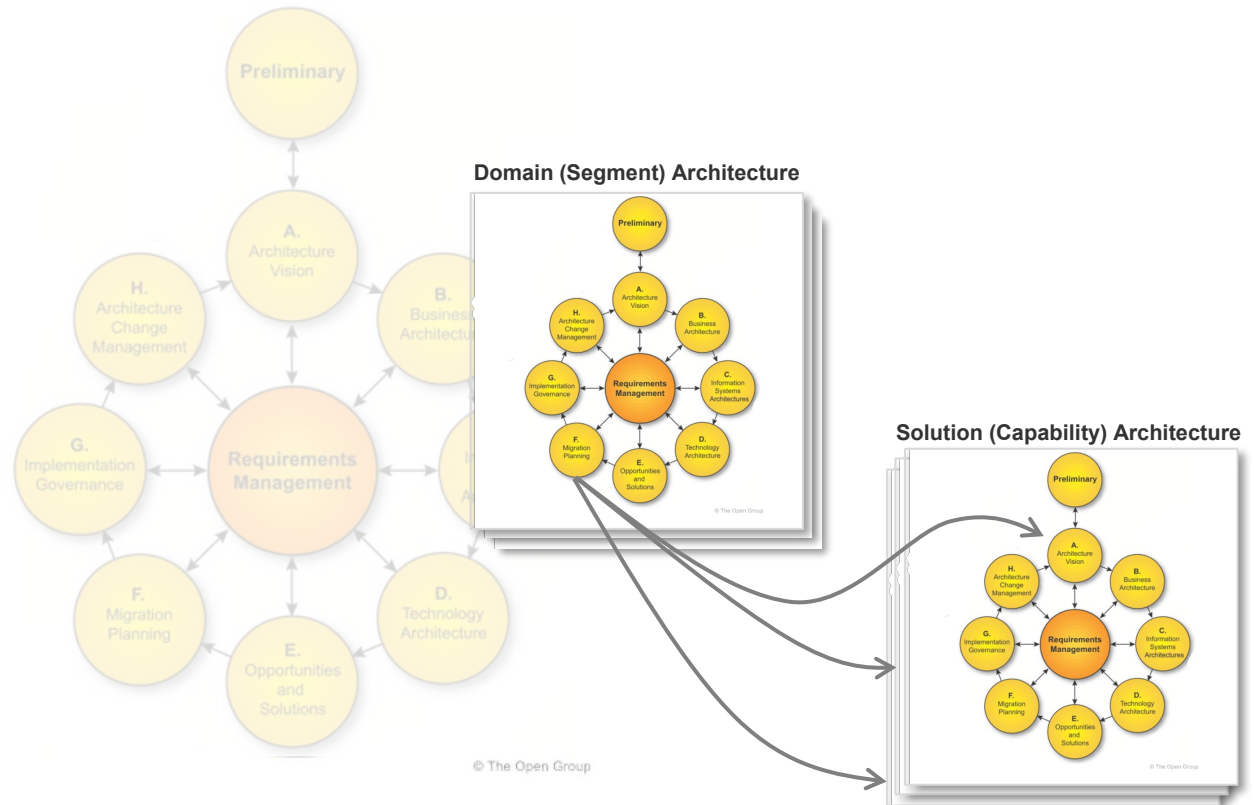
Methodology

Architecture Process

An **Architecture Process** is a **structured methodology** within an **Architecture Framework** (e.g., TOGAF ADM) that **provides** a generic **procedure** for **developing, managing** and **governing** the **architecture** of an **organisation**.

... a corresponding **perspective** at **architecture segment level** **focuses** on the corresponding **domain as a whole** (e.g. private banking or investment banking at a bank) ...

... and an **architecture perspective** at **capability level** **focusses** on an **individual solution** (e.g. an online banking application in private banking).



Methodology

Architecture Process and View Model

Architecture Process and **Architecture View Model** are two sides of the same coin: while the **process** describes the **procedure** for **elaborating results**, the **view model** provides **templates** for these **results'** manifestation.

Just as there is no
hike ...

... there is no
architecture
process, ...



... without leaving
footprints behind, ...

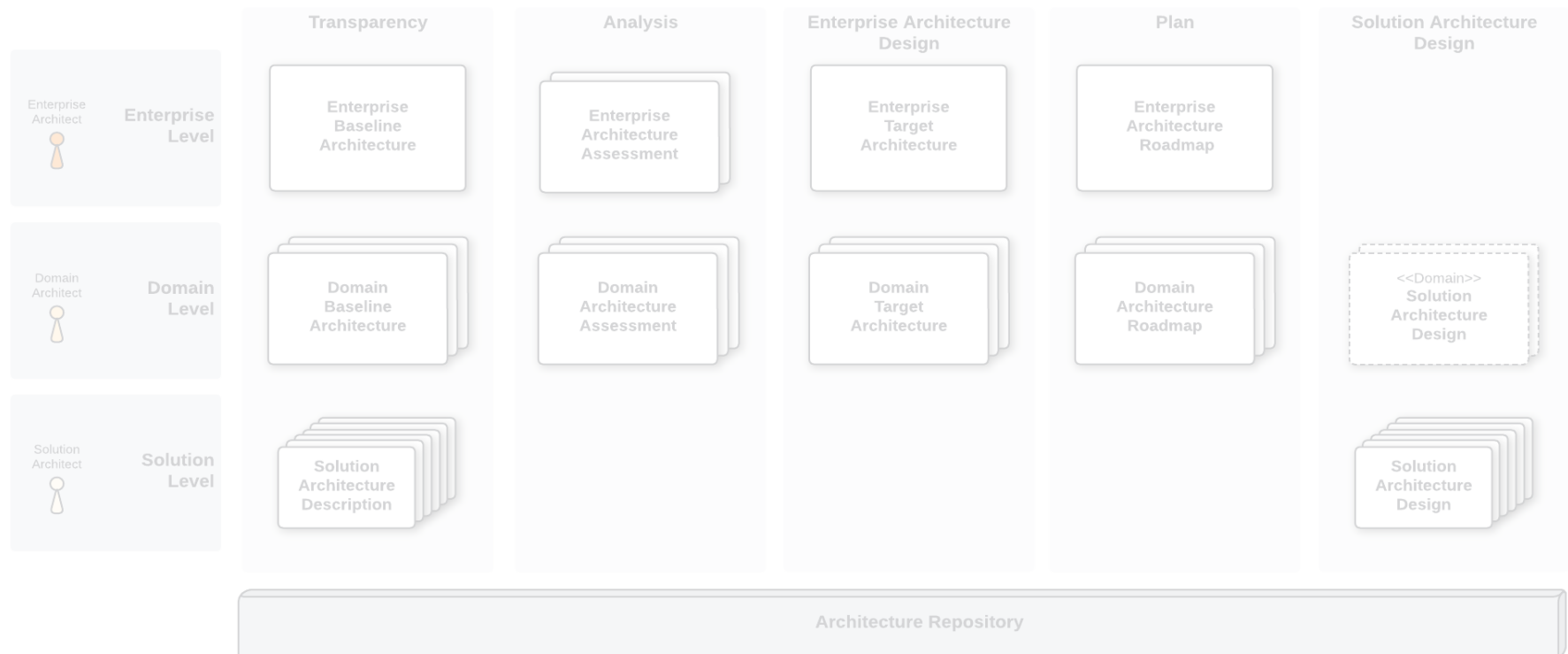
... without a
corresponding
architecture view
model.



Methodology

End-2-End Example

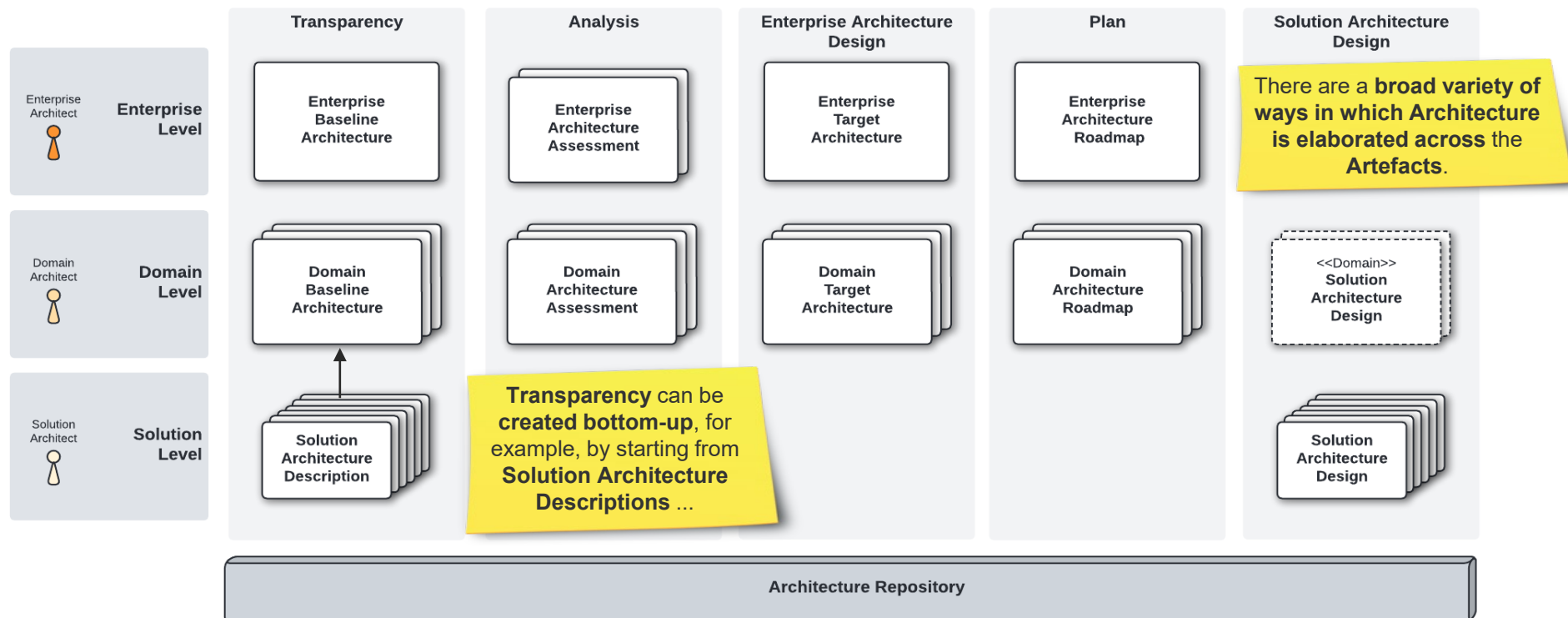
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

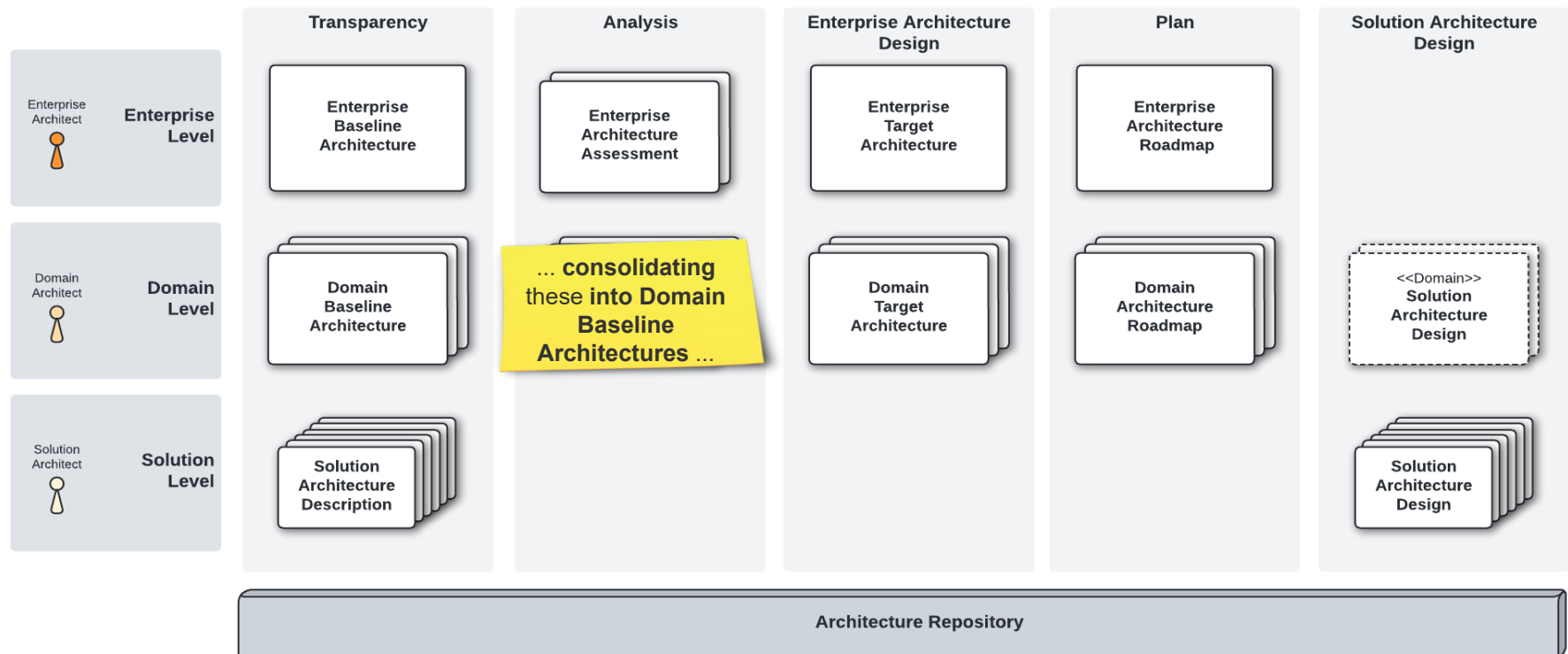
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

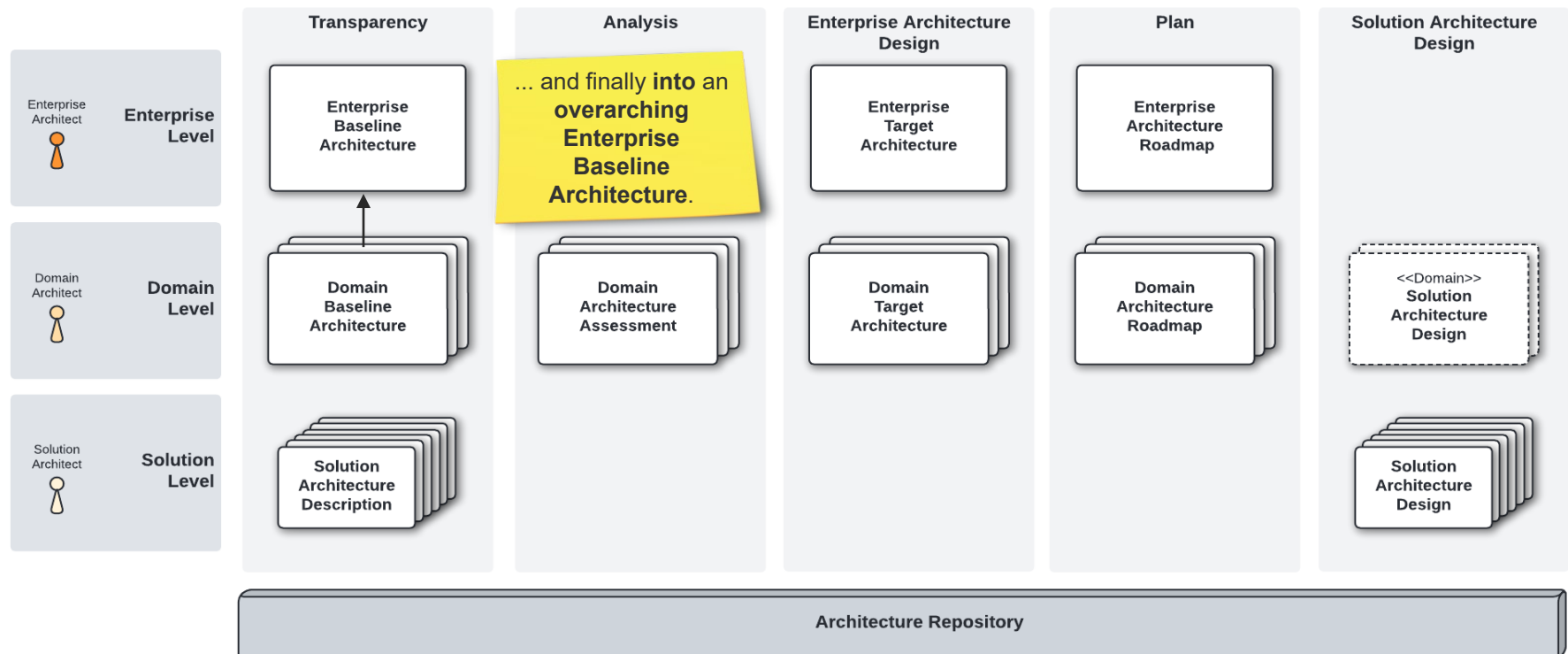
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

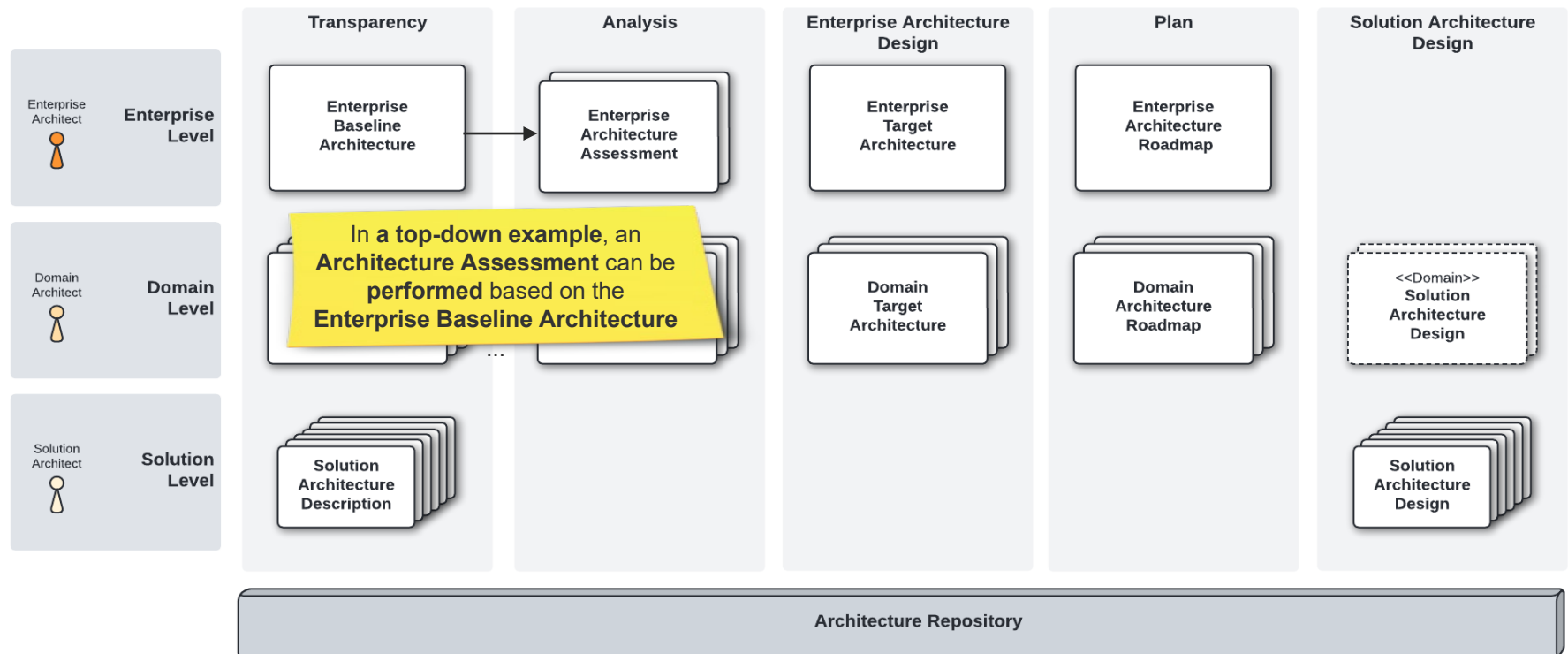
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

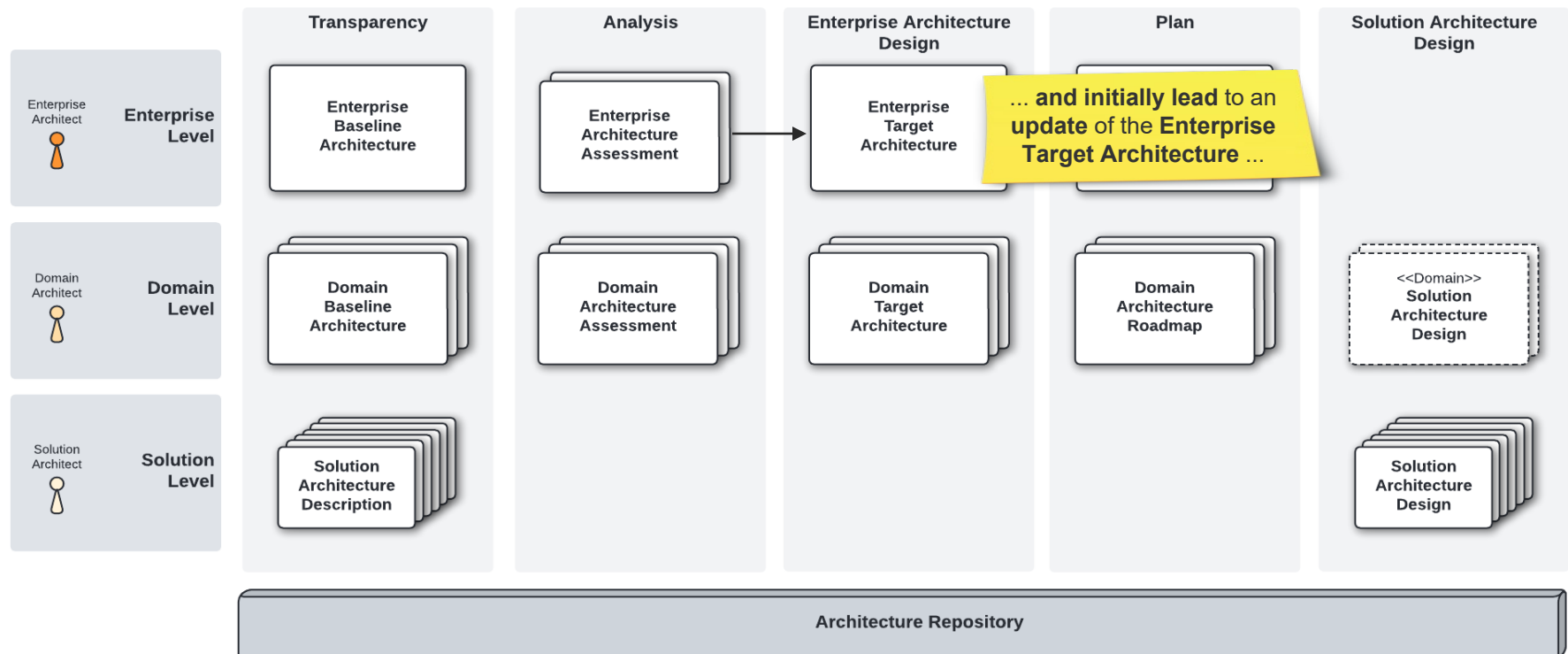
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

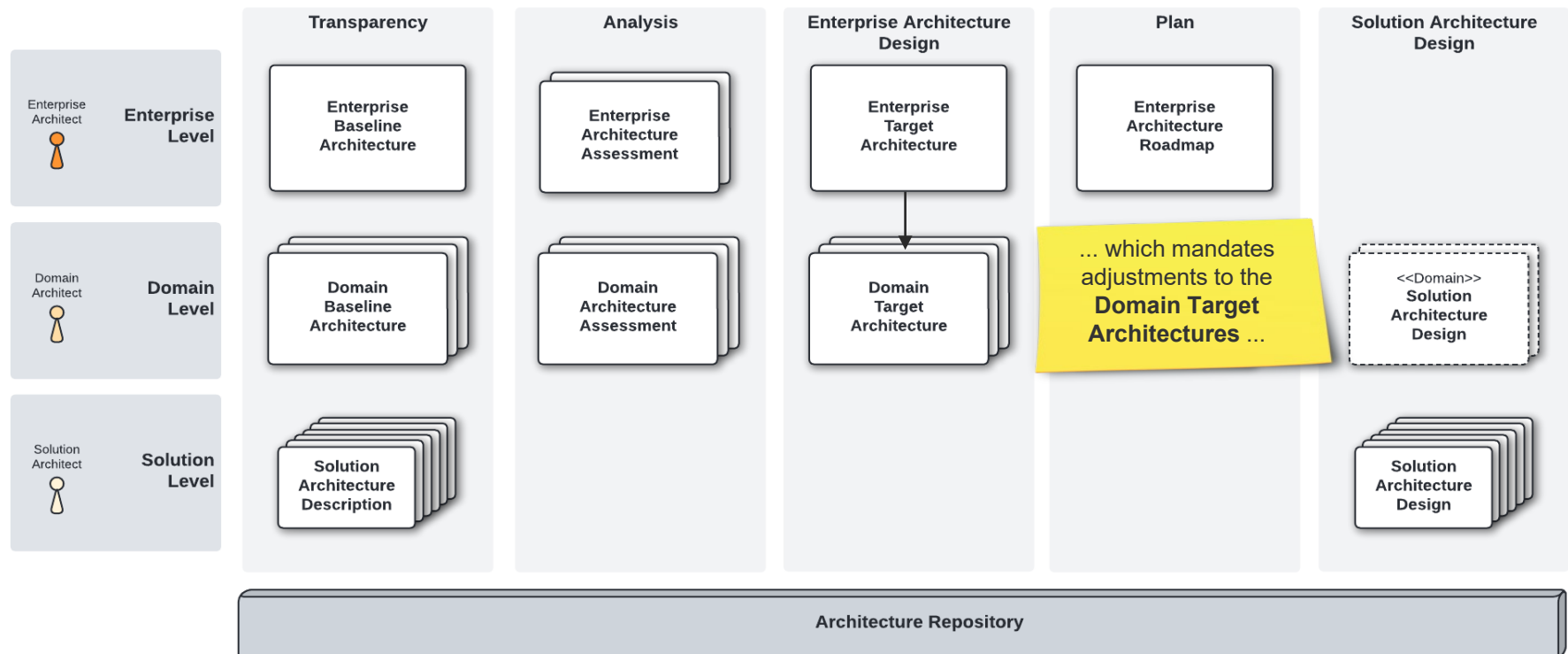
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

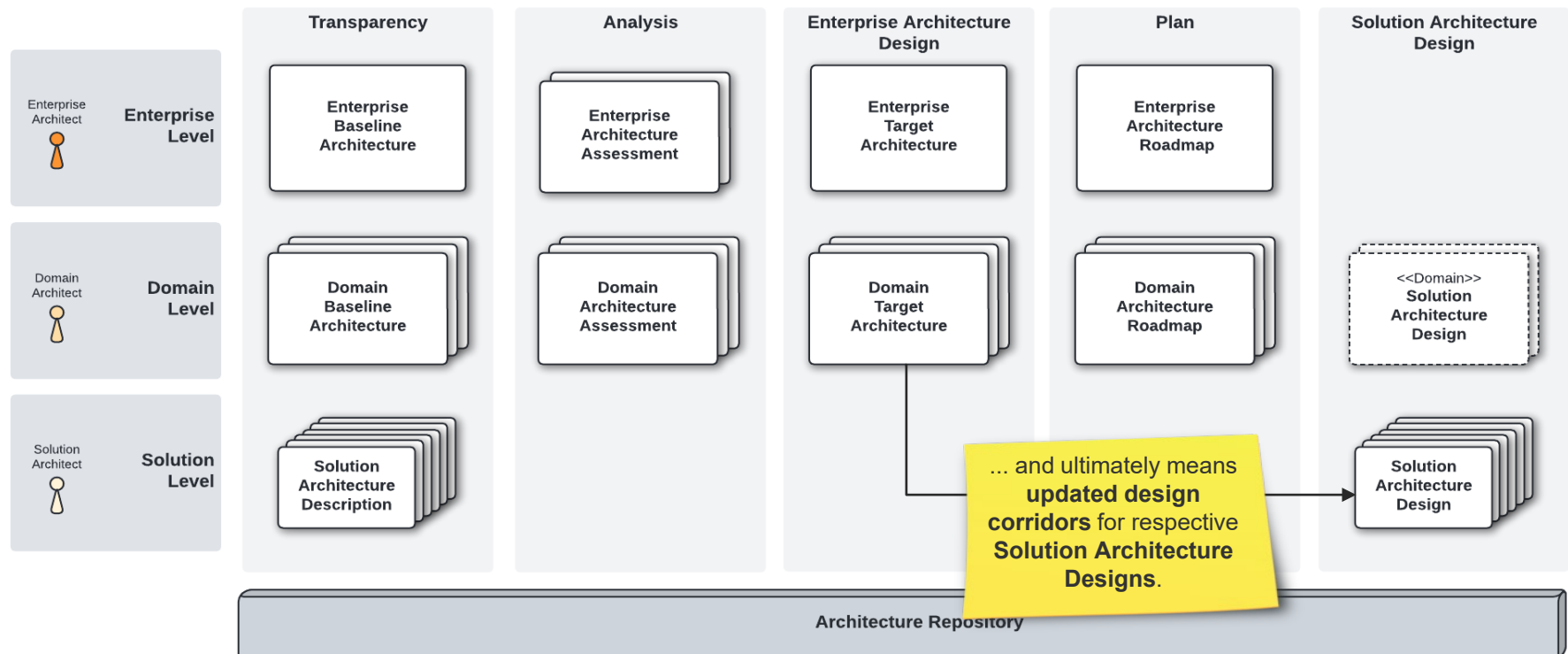
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

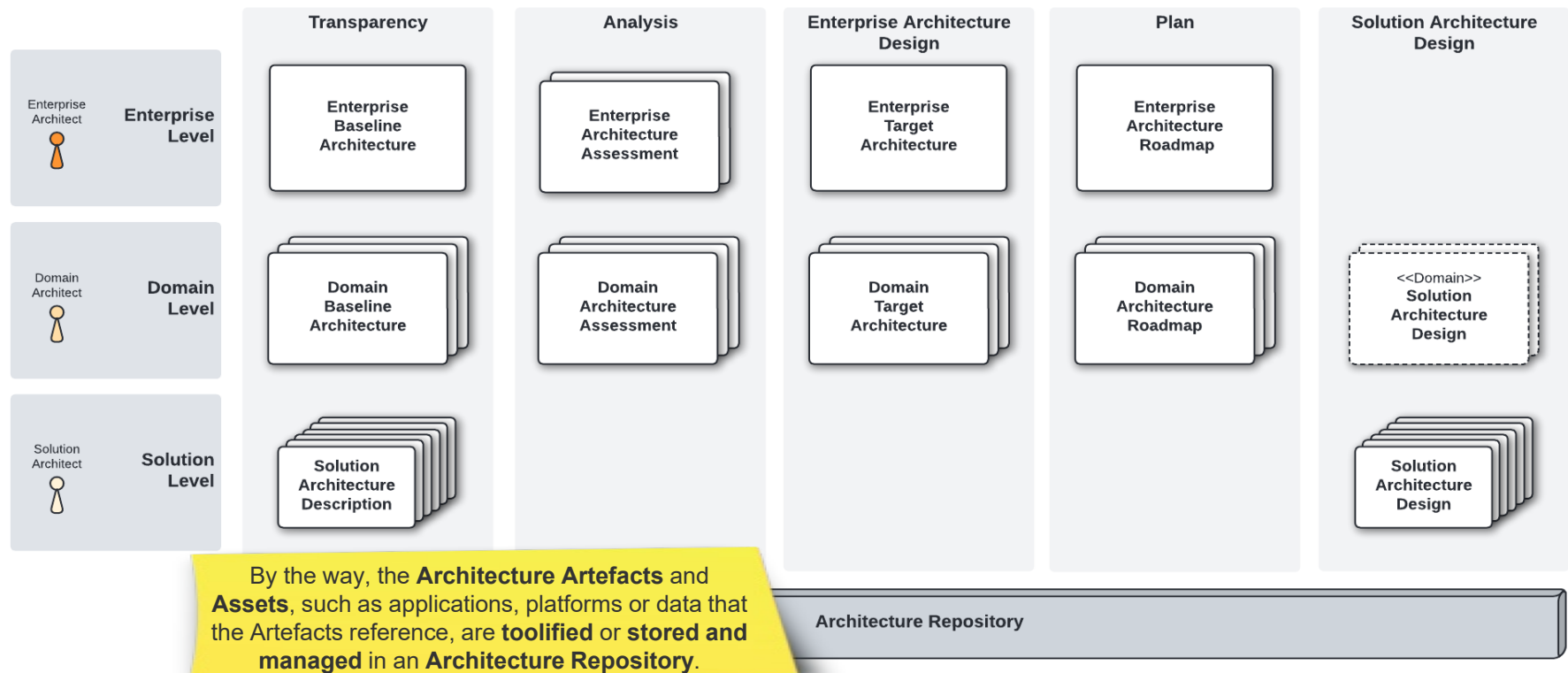
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

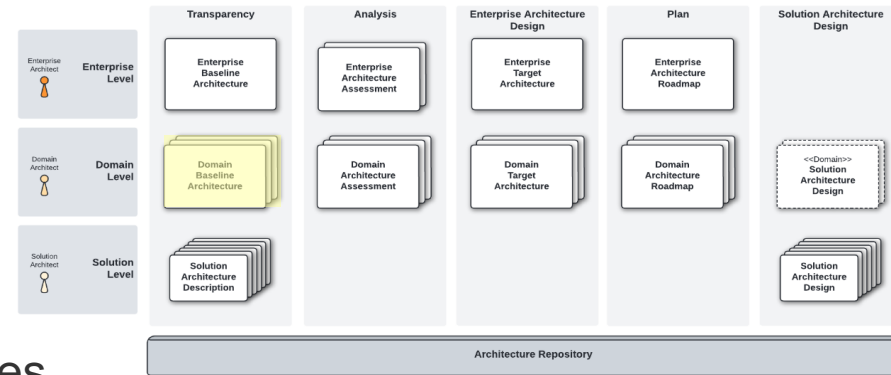
The following slides present a **concrete scenario** covering the **3 architecture levels** and **end-to-end**, to **demonstrate architecture development holistically**.



Methodology

End-2-End Example

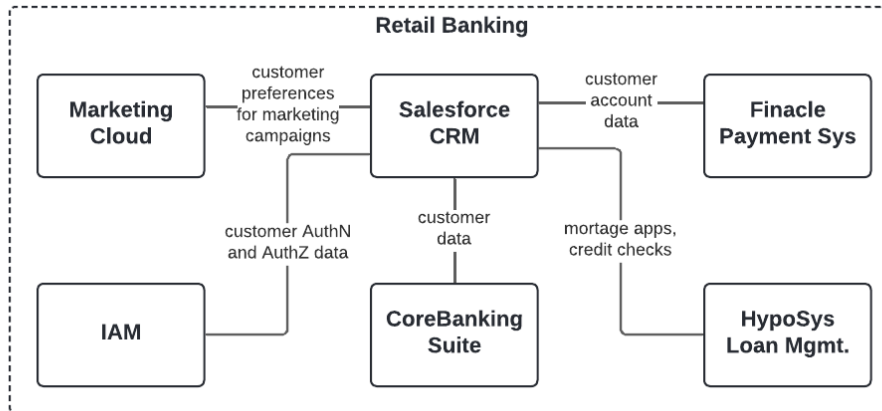
Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



We **start our example** with the **Transparency Activity** at **Domain Level**.

The **Domain Architect** of the **Retail Banking domain** maintains several **views** and artefacts as part of his **Domain Baseline Architecture**.

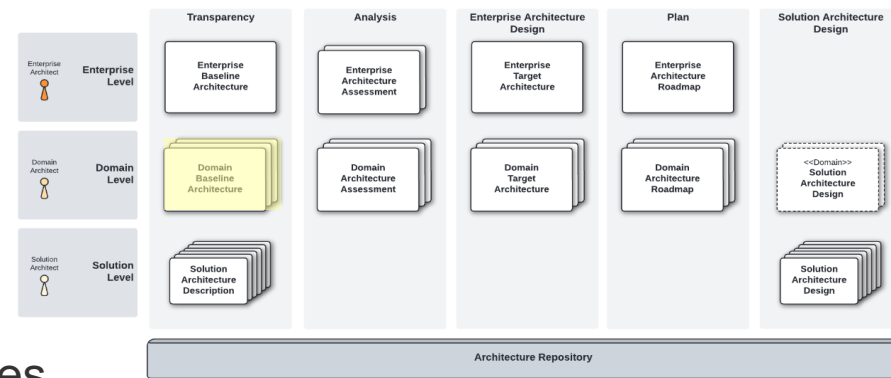
One of these **views captures** the domain's most **important Applications** and **crucial Information Objects** that are **exchanged between the Applications**.



Methodology

End-2-End Example

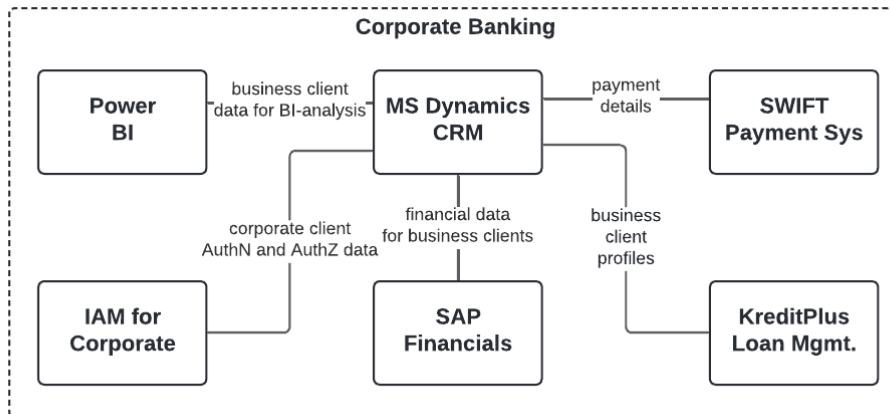
Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Next we **step into the shoes of the Corporate Banking Domain Architect ...**

The **Domain Architect** also **maintains a set of views** as part of his **Domain Baseline Architecture...**

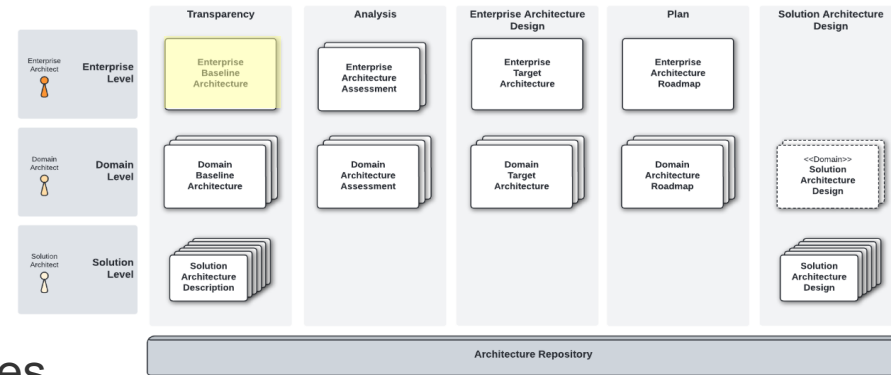
... including a view that also captures Applications, Information Objects and exchanges between Applications within the Domain.



Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



When we **move** to the **Enterprise Architecture** level, we **recognize** that **Enterprise Architects** manage **cross-domain views**.

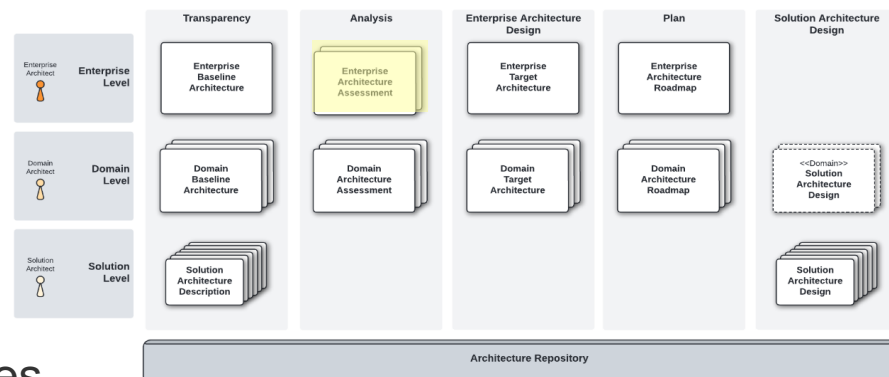
Capability	Customer Relationship Management	Loan Management	Financial Planning	Payment Processing	Data Analysis	Identity & Access Mgmt.
Domain						
Retail Banking	Salesforce CRM	HyperSys Loan Mgmt.	CoreBanking Suite	Finacle Payment Sys	Marketing Cloud	IAM
Corporate Banking	MS Dynamics CRM	KreditPlus Loan Mgmt.	SAP Financials	SWIFT Payment Sys	Power BI	IAM for Corporate

In this view, the **Enterprise Architect** looks at which **Applications** (crossings) in the **two Domains** of Retail Banking and Corporate Banking (rows) **realize** or address which **Business Capabilities** (columns).

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



We remain at the **Enterprise Architecture Level**, but move on to the **Analysis stage**.

While the Transparency Stage is free of judgements, the Analyse Stage includes analyses with resulting assessments and evaluations.

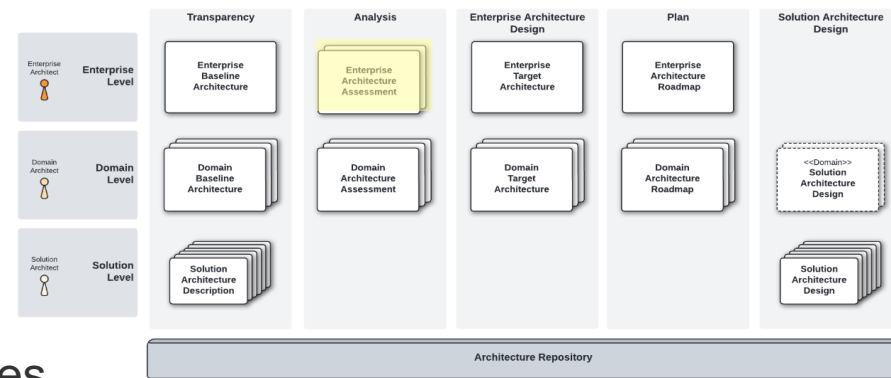
One of **many** possible **analyses** looks at **consolidation potentials**, which – as shown in the matrix on the left – are **further differentiated** using **colour-coding**.

Capability	Customer Relationship Management	Loan Management	Financial Planning	Payment Processing	Data Analysis	Identity & Access Mgmt.
Domain						
Retail Banking	Salesforce CRM	HyperSys Loan Mgmt.	CoreBanking Suite	Finacle Payment Sys	Marketing Cloud	IAM
Corporate Banking	MS Dynamics CRM	KreditPlus Loan Mgmt.	SAP Financials	SWIFT Payment Sys	Power BI	IAM for Corporate
Replication	Coordination					
Diversification	Unification					

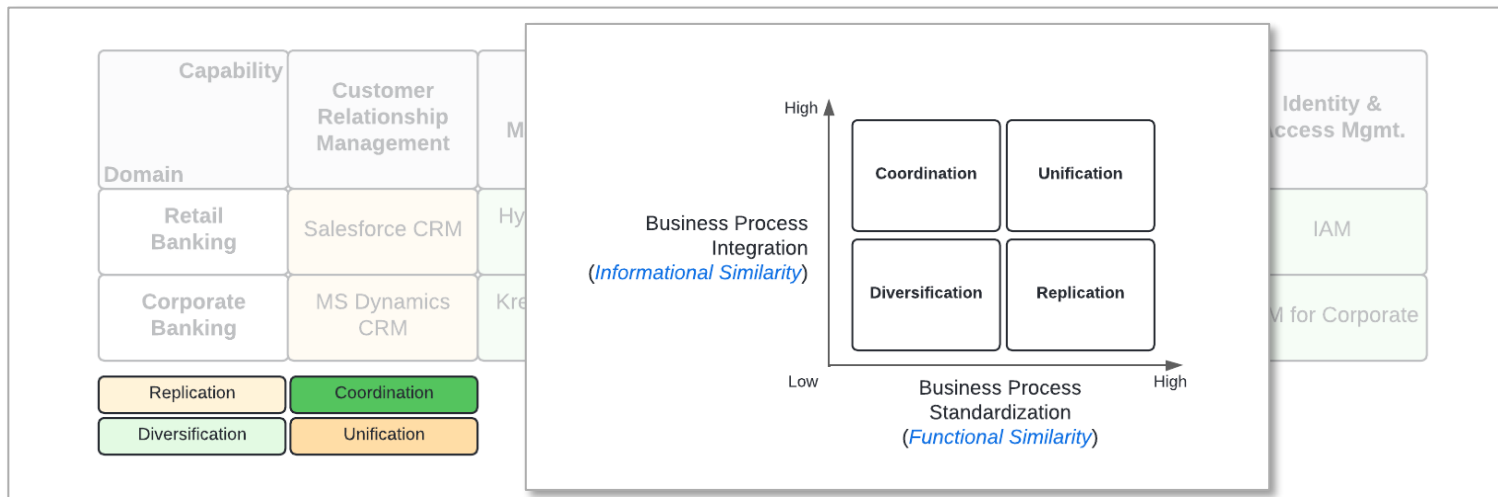
Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



The **metric** behind the investigation of **consolidation potentials** considers on the one hand the **similarity of processes** (Functionality) that are **realized by different applications** ...



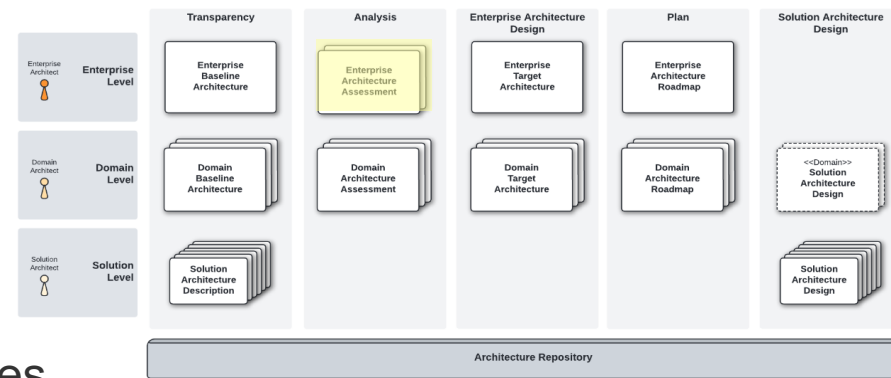
... and on the other hand the **similarity of the Data Structures and Sets** that are **covered by different applications**.

Replication, for example, means that **two or more applications** have **similar algorithmic structure** (i.e., **process similarity**), but **differ regarding their data sets**.

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Two **types of consolidation opportunities** were discovered (i.e., **Replication** and **Unification**).

Capability	Customer Relationship Management	Loan Management	Financial Planning	Payment Processing	Data Analysis	Identity & Access Mgmt.
Domain						
Retail Banking	Salesforce CRM	HyperSys Loan Mgmt.	CoreBanking Suite	Finacle Payment Sys	Marketing Cloud	IAM
Corporate Banking	MS Dynamics CRM	KreditPlus Loan Mgmt.	SAP Financials	SWIFT Payment Sys	Power BI	IAM for Corporate
Replication	Coordination					
Diversification	Unification					

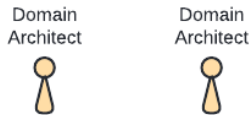
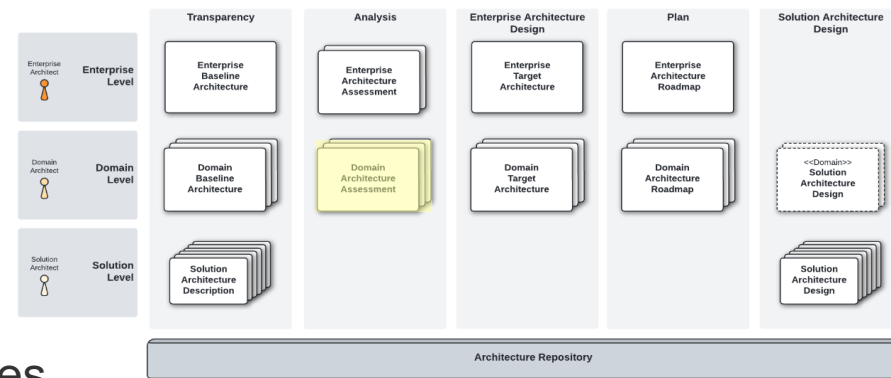
In a first pass the **Enterprise Architect** is **concentrating** on the **Replication opportunities** – that is, any **consolidation options** relating to the two applications **Salesforce CRM** and **MS Dynamics CRM**.

The **Enterprise Architect** **passes the ball back** to the two **Domain Architects** ...

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



... and the **two Domain Architects** take the ball accordingly.

Criteria	Multi Tenancy
CRM Application	
Salesforce CRM	4
MS Dynamics CRM	3

Salesforce CRM is renowned for its robust multi-tenancy, enabling efficient management of different customer groups (e.g., retail vs. corporate).

Microsoft Dynamics CRM also supports multi-tenancy but requires additional customization for specific needs.

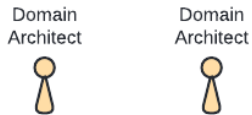
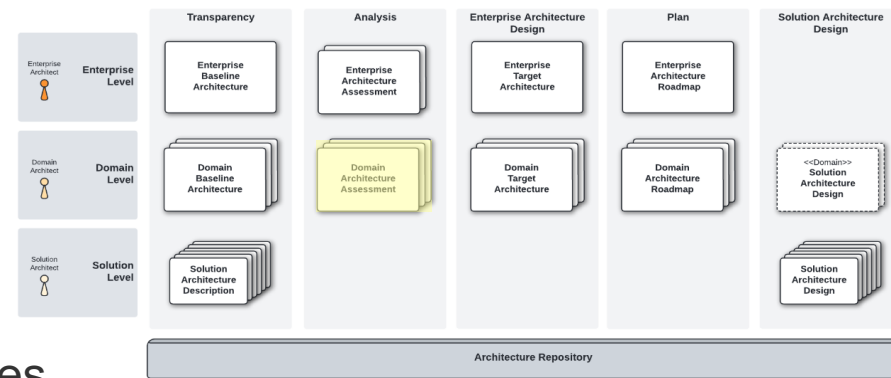
The **first criterion** considered is the **Multi-Tenancy capability** of the two **applications**, to address the **different data sets** with a **single application**.

They come together to **analyse** the **possibilities** for **replacing** the **one CRM application** with the other and thus **combining two apps** into one.

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Criteria	Multi Tenancy	Scalability
CRM Application		
Salesforce CRM	4	4
MS Dynamics CRM	3	2

Salesforce CRM is renowned for its robust multi-tenancy, enabling efficient management of different customer groups (e.g., retail vs. corporate).

Microsoft Dynamics CRM also supports multi-tenancy but requires additional customization for specific needs.

Salesforce CRM is highly scalable, suitable for global operations.

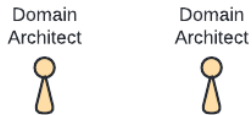
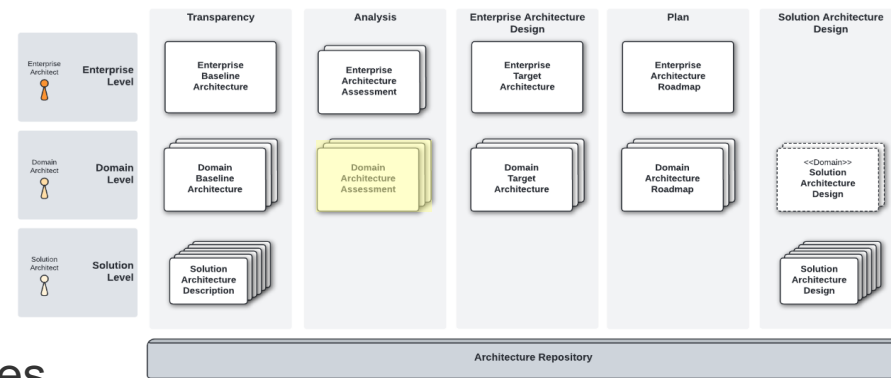
The scalability of Microsoft Dynamics CRM for expanding corporate business needs is limited.

The **next criterion** considered when looking at consolidation options is the **scalability of the two systems**, where **Salesforce CRM** has a **strong advantage over MS Dynamics CRM**.

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Criteria	Multi Tenancy	Scalability	Licensing Model
CRM Application			
Salesforce CRM	4	4	2
MS Dynamics CRM	3	2	2

The **license models** of the two applications **differ only slightly**, so they both **equally rated** in the evaluation.

Salesforce CRM is renowned for its robust multi-tenancy, enabling efficient management of different customer groups (e.g., retail vs. corporate).

Microsoft Dynamics CRM also supports multi-tenancy but requires additional customization for specific needs.

Salesforce CRM is highly scalable, suitable for global operations.

The scalability of Microsoft Dynamics CRM for expanding corporate business needs is limited.

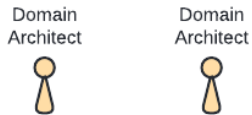
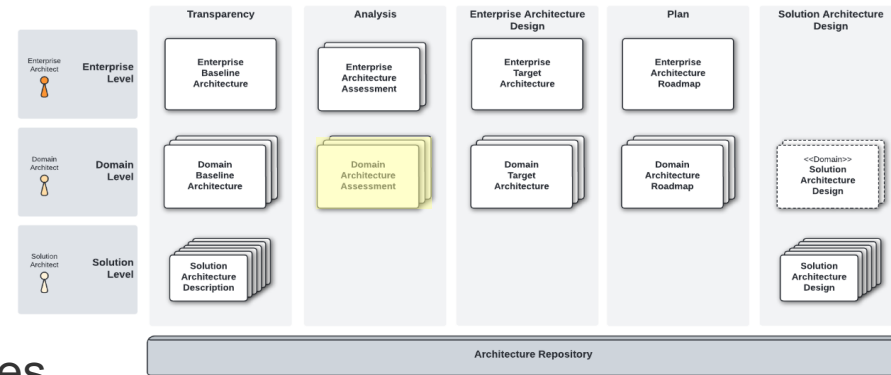
Salesforce CRM follows a subscription-based licensing model, which incurs relatively high costs for additional users and customization.

Microsoft Dynamics CRM also offers flexible licensing and is similarly cost-effective as Salesforce CRM.

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Criteria	Multi Tenancy	Scalability	Licensing Model	Data Privacy & Compliance
CRM Application				
Salesforce CRM	4	4	2	3
MS Dynamics CRM	3	2	2	3

Also in terms of their **data privacy and compliance capabilities**, both applications differ only marginally.

Salesforce CRM is renowned for its robust multi-tenancy, enabling efficient management of different customer groups (e.g., retail vs. corporate).

Microsoft Dynamics CRM also supports multi-tenancy but requires additional customization for specific needs.

Salesforce CRM is highly scalable, suitable for global operations.

The scalability of Microsoft Dynamics CRM for expanding corporate business needs is limited.

Salesforce CRM follows a subscription-based licensing model, which incurs relatively high costs for additional users and customization.

Microsoft Dynamics CRM also offers flexible licensing and is similarly cost-effective as Salesforce CRM.

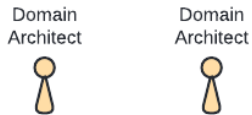
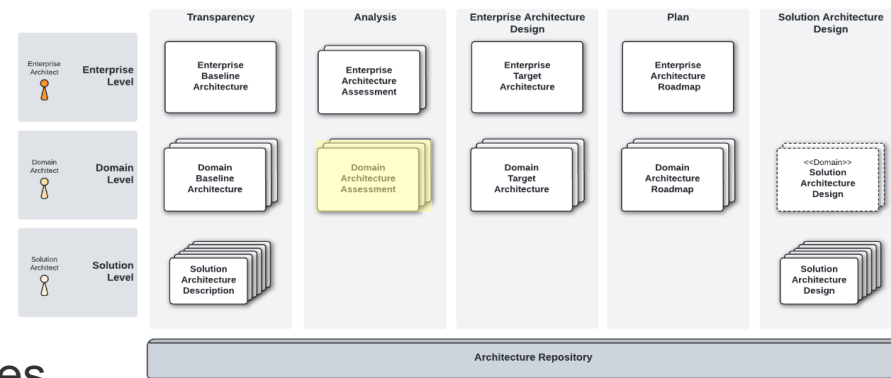
Salesforce CRM adheres to strict privacy regulations like GDPR and offers advanced security features.

Microsoft Dynamics CRM meets the same compliance standards, especially for business client data.

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Criteria	Multi Tenancy	Scalability	Licensing Model	Data Privacy & Compliance	Data Migration & Integration
CRM Application					
Salesforce CRM	4	4	2	3	2
MS Dynamics CRM	3	2	2	3	2

The **same** applies to the **Data Migration and Integration features** of both **applications**: only **marginal differences** were determined.

Salesforce CRM is renowned for its robust multi-tenancy, enabling efficient management of different customer groups (e.g., retail vs. corporate).

Microsoft Dynamics CRM also supports multi-tenancy but requires additional customization for specific needs.

Salesforce CRM is highly scalable, suitable for global operations.

The scalability of Microsoft Dynamics CRM for expanding corporate business needs is limited.

Salesforce CRM follows a subscription-based licensing model, which incurs relatively high costs for additional users and customization.

Microsoft Dynamics CRM also offers flexible licensing and is similarly cost-effective as Salesforce CRM.

Salesforce CRM adheres to strict privacy regulations like GDPR and offers advanced security features.

Microsoft Dynamics CRM meets the same compliance standards, especially for business client data.

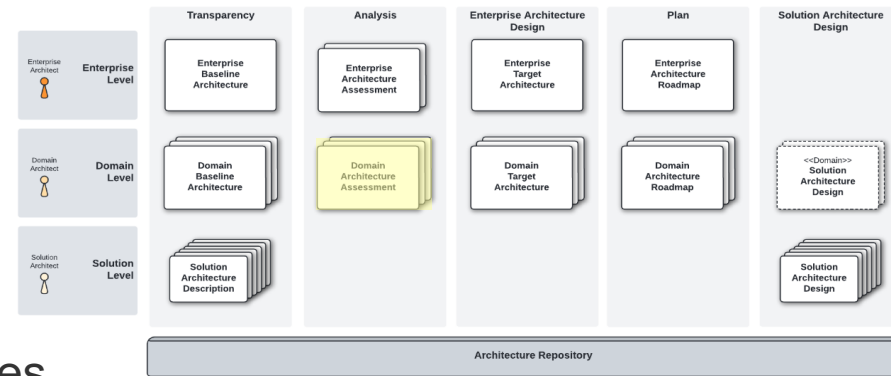
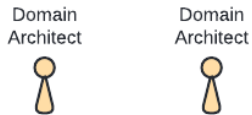
If Salesforce CRM is chosen, a detailed migration strategy for transferring business client data from Microsoft Dynamics CRM is crucial.

This could have downstream impacts on systems like KreditPlus and SAP Financials, which are tightly integrated with Microsoft Dynamics.

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



In terms of **User Acceptance**, **Salesforce CRM** is again slightly ahead of **MS Dynamics CRM**.

After doing the math, both the **Domain Architects** and their evaluation team conclude that **Salesforce CRM** is better positioned to handle the **CRM needs** and **data across both Domains**.

Criteria	Multi Tenancy	Scalability	Licensing Model	Data Privacy & Compliance	Data Migration & Integration	User Acceptance & Training
CRM Application						
Salesforce CRM	4	4	2	3	2	3
MS Dynamics CRM	3	2	2	3	2	2

Salesforce CRM is renowned for its robust multi-tenancy, enabling efficient management of different customer groups (e.g., retail vs. corporate).

Microsoft Dynamics CRM also supports multi-tenancy but requires additional customization for specific needs.

Salesforce CRM is highly scalable, suitable for global operations.

The scalability of Microsoft Dynamics CRM for expanding corporate business needs is limited.

Salesforce CRM follows a subscription-based licensing model, which incurs relatively high costs for additional users and customization.

Microsoft Dynamics CRM also offers flexible licensing and is similarly cost-effective as Salesforce CRM.

Salesforce CRM adheres to strict privacy regulations like GDPR and offers advanced security features.

Microsoft Dynamics CRM meets the same compliance standards, especially for business client data.

If Salesforce CRM is chosen, a detailed migration strategy for transferring business client data from Microsoft Dynamics CRM is crucial.

This could have downstream impacts on systems like KreditPlus and SAP Financials, which are tightly integrated with Microsoft Dynamics.

Consider the user experience and adoption of Salesforce CRM by corporate banking staff.

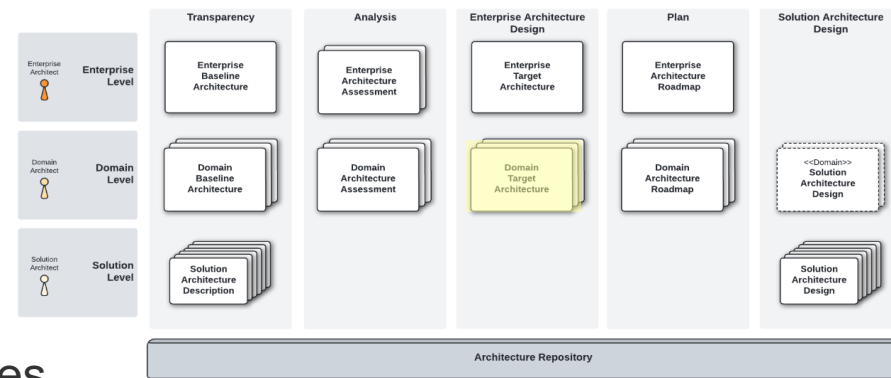
If Salesforce is implemented across both domains, a comprehensive training program will be needed.

Together with the **Enterprise Architect**, both **Domain Architects** decide to **replace MS Dynamics CRM** with **Salesforce CRM**.

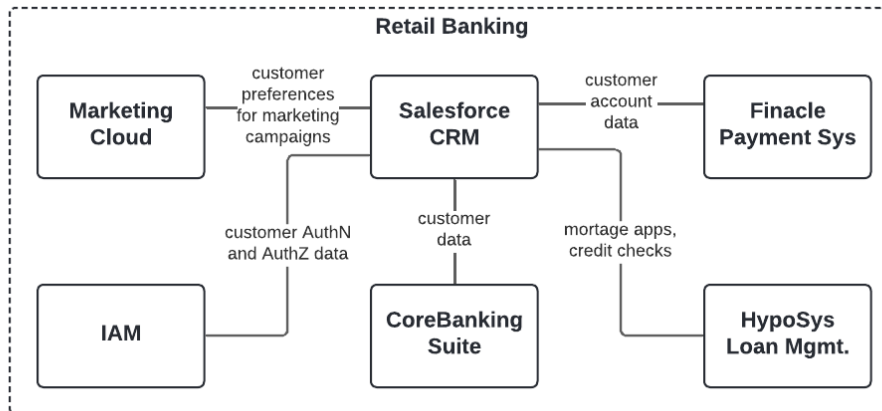
Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



The **decision** that was taken lately **must first be reflected** in all impacted **Domain Target Architectures** and the **consequences** associated with this change **must be further analyzed**.

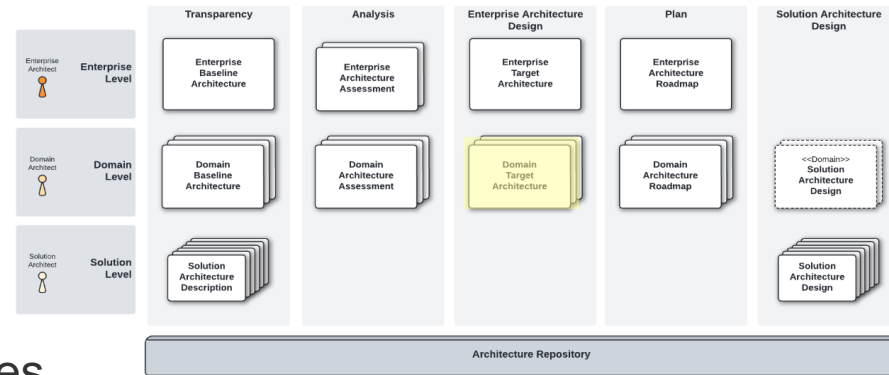


Since **no changes** are **pending** in the **Retail Banking** domain, the corresponding **Architect** does **not need to update** its **Target Architecture**.

Methodology

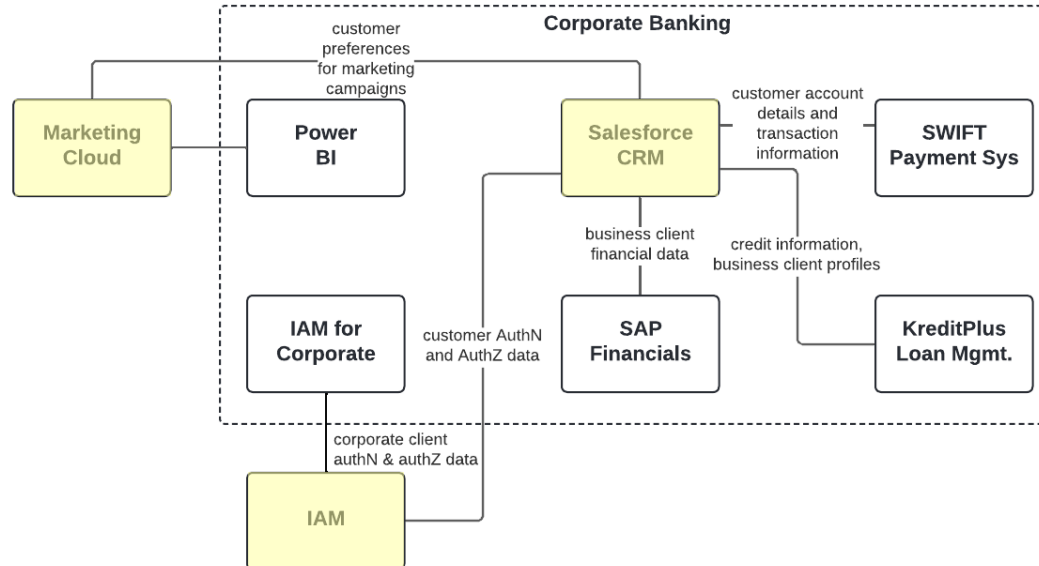
End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



However, in the **Corporate Banking** domain ...

... the responsible **Architect** performs the required update to his **Domain Target Architecture**.



Methodology

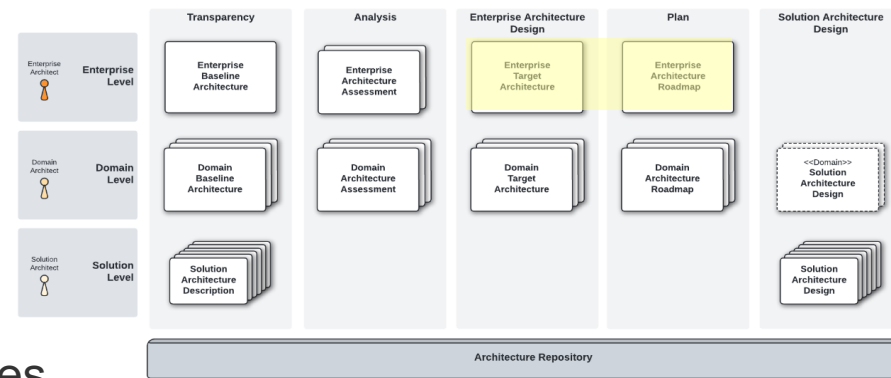
End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Project Portfolio (Bank)

Project	Category	Objectives and Goals	Priority Level	Resource Allocation	Risk Assessment	Timeline and Milestones	Key Stakeholders	Status
CRM Consolidation	Strategic	Reducing redundancies and improving efficiency in customer management.	High	CHF 2 million, 10 IT experts	High	Start: Q4 2024, End: Q4 2026	CTO, Head of Digital Banking	Proposed
Payment System Modernization	Operational	Enhancing payment processing efficiency.	Medium	CHF 0.5 million	Medium	Start: Q2 2025, End: Q4 2025	Head of Payment System	Proposed
Cloud Migration for HypoSys	Operational	Scaling loan management for future requirements.	Medium	CHF 0.5 million	Medium	Start: Q2 2025, End: Q4 2025	Sub-Project of CRM Consolidation (similar Stakeholders)	Proposed
Branch Network Consolidation	Operational	Reduce operational costs by closing underperforming	Medium	CHF 3 million, operations team	High	Start: Q2 2024, End: Q3 2024	Head of Retail Operations	In Progress
Legacy IT System Decommissioning	Operational	Phase out outdated systems and migrate to the cloud	High	CHF 2 million, IT staff	High	Start: Q1 2024, End: Q3 2024	CTO	In Progress



In a next step the **Enterprise Architect**, together **with the Domain Architects**, supports the **Project Portfolio Manager** ...

... in **correctly positioning** the new **project** via which the **application consolidation** is to be implemented.

The **consolidation project** has been **given high priority** because it is expected to **make a significant contribution** to **reducing total operating costs**.

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Project Proposal – CRM Consolidation

Project Overview

Purpose: Consolidation of CRM Systems for Retail and Corporate Banking.

Scope: Migration of business client data from Microsoft Dynamics CRM to Salesforce CRM, along with interface updates for related systems (e.g., SAP Financials, KreditPlus).

Business Value

- Full migration of business client data without data loss, operational Salesforce CRM with complete functionality, adherence to all privacy regulations.

Risk Assessment

- Data migration complexity.
- Potential disruptions during migration.
- High licensing and customization costs for Salesforce.

Cost Estimate

- CHF 2 Million.

Schedule

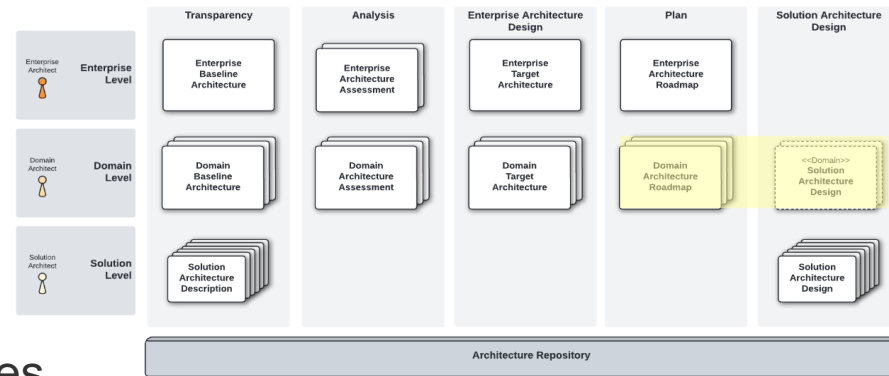
- Start in Q4 2024, with completion in Q2 2026.
- Integration Testing finished: Q3 2025, etc.

Deliverables

- Consolidated CRM system (based on Salesforce)
- Decommissioned MS Dynamics CRM

Skills and Resources

- **Resources:** IT architects, data migration specialists, business analysts, and compliance officers.



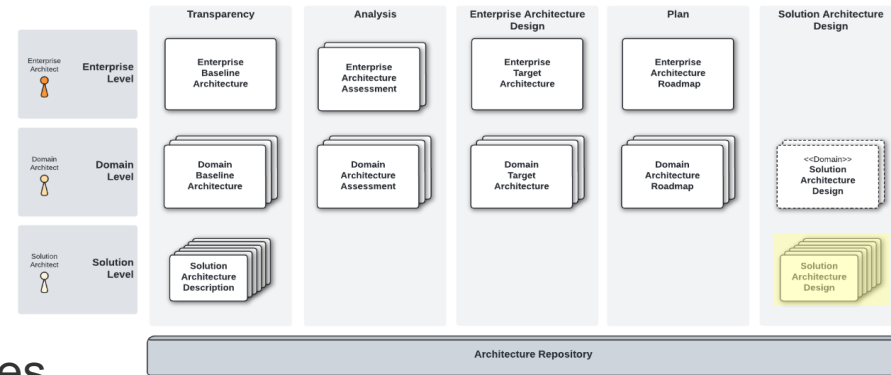
From **project portfolio management**, the **green light** has been **given** for **kickstarting** the **CRM Consolidation project** ...

...and **both Domain Architects** are **ensuring** that their **Architectural views and conditions** are **sufficiently reflected** in the **charter** of the **CRM consolidation project**.

Methodology

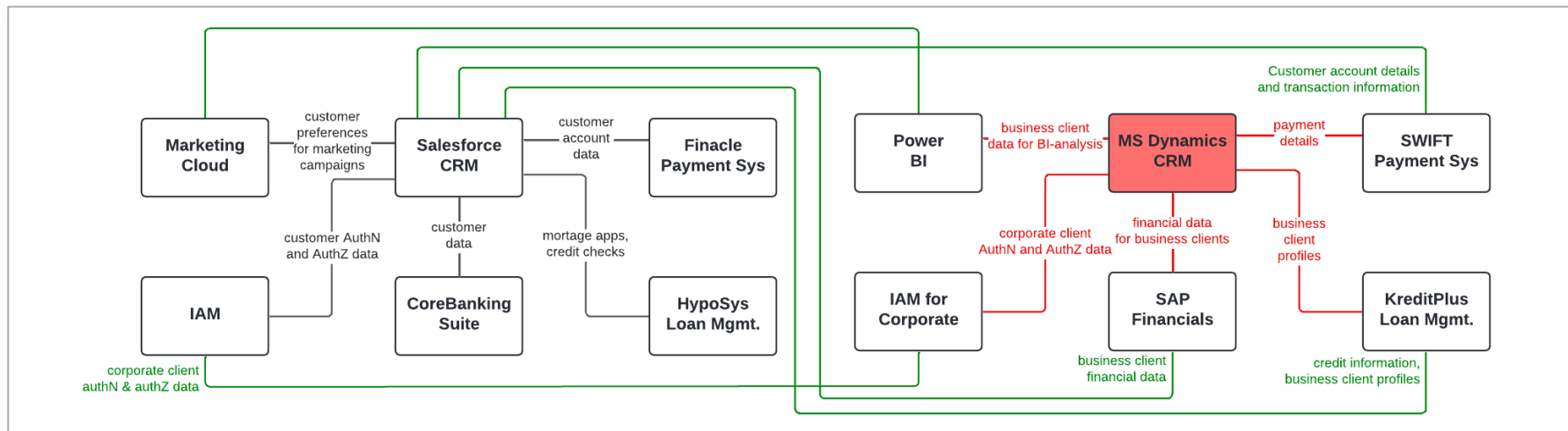
End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Embedded in the CRM consolidation project,
a **Solution Architect** is now **elaborating a detailed design** of the **consolidated Solution**.

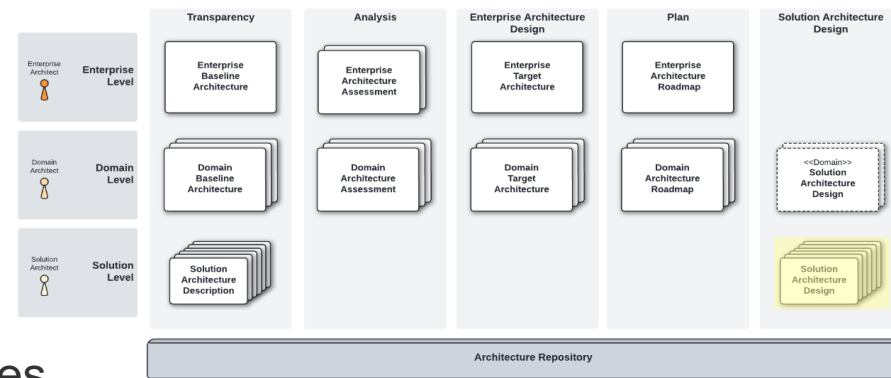
The **Solution Architect** collaborates with the **Domain Architect** to ensure that the **domain context** and **overarching architecture objective** are adequately **considered** in his design.



Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.

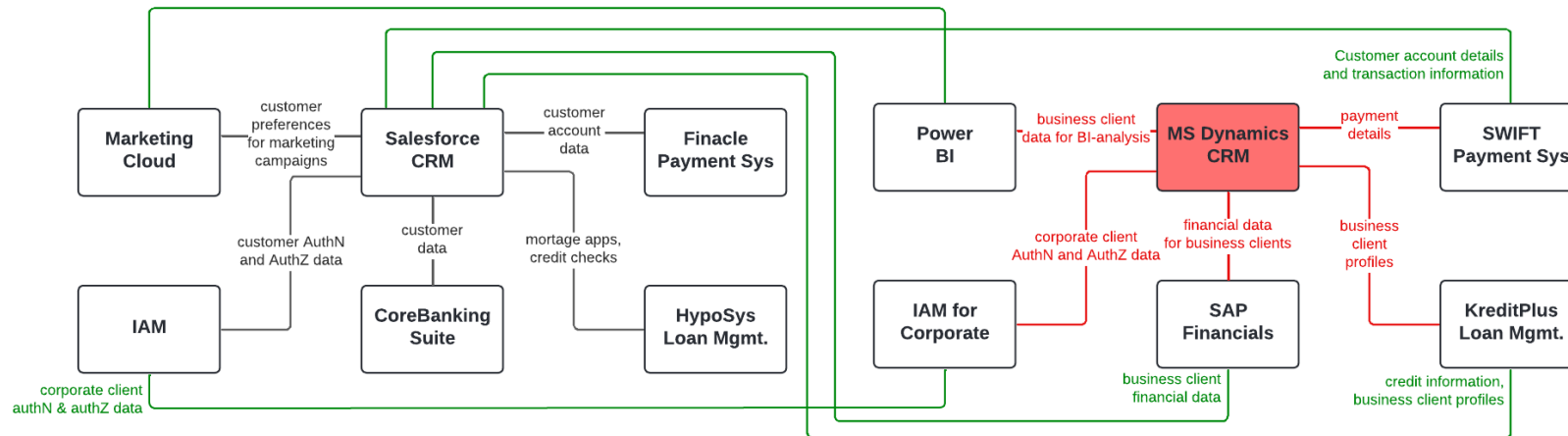


There are a **lot of aspects** that **Solution Architects** needs to **consider** when **performing** their **design work**.

For example, **designing (filter logic), updating, removal and testing of interfaces** between Salesforce CRM and SAP Financials, as well as other affected systems...

... but also designing a **Migration Plan** that **ensures** that the **CRM data** of the **Corporate Banking domain** is **seamlessly transferred** to **Salesforce CRM** ...

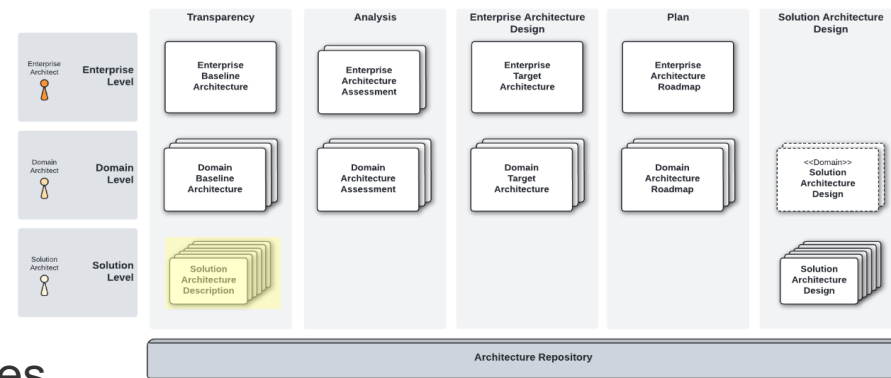
... or finally **ensuring** the **complete dismantling** of **MS Dynamics CRM** at the end of the consolidation journey.



Methodology

End-2-End Example

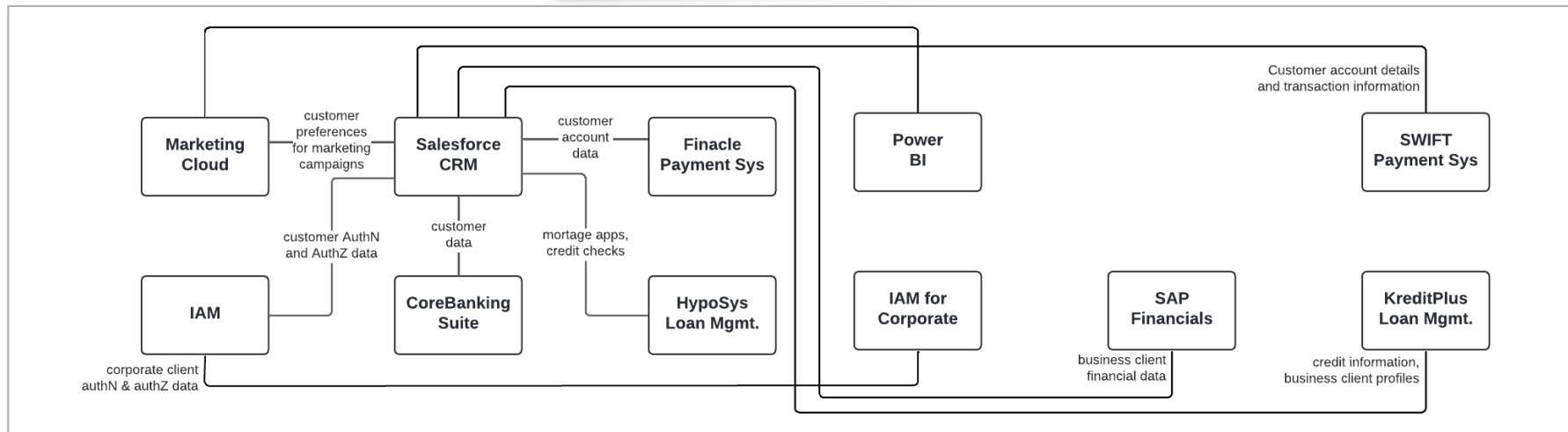
Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Once the **recent design** is **finalized** and **implemented**, a new **Solution Architecture description** is available.

The new design describes a recent **change** to the existing **Enterprise Landscape**.

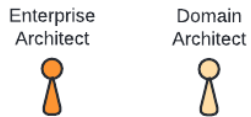
This must now be reflected in the **Baseline Architecture** in both the **Architecture Repository** and the **artifacts** at the **Domain Architecture** and **Enterprise Architecture** levels.



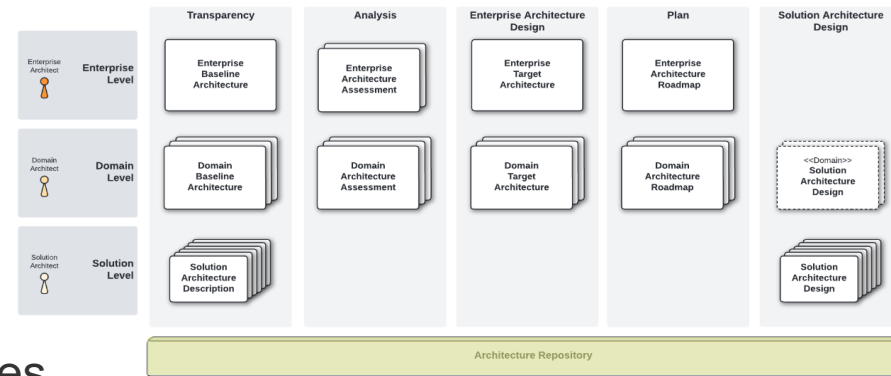
Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



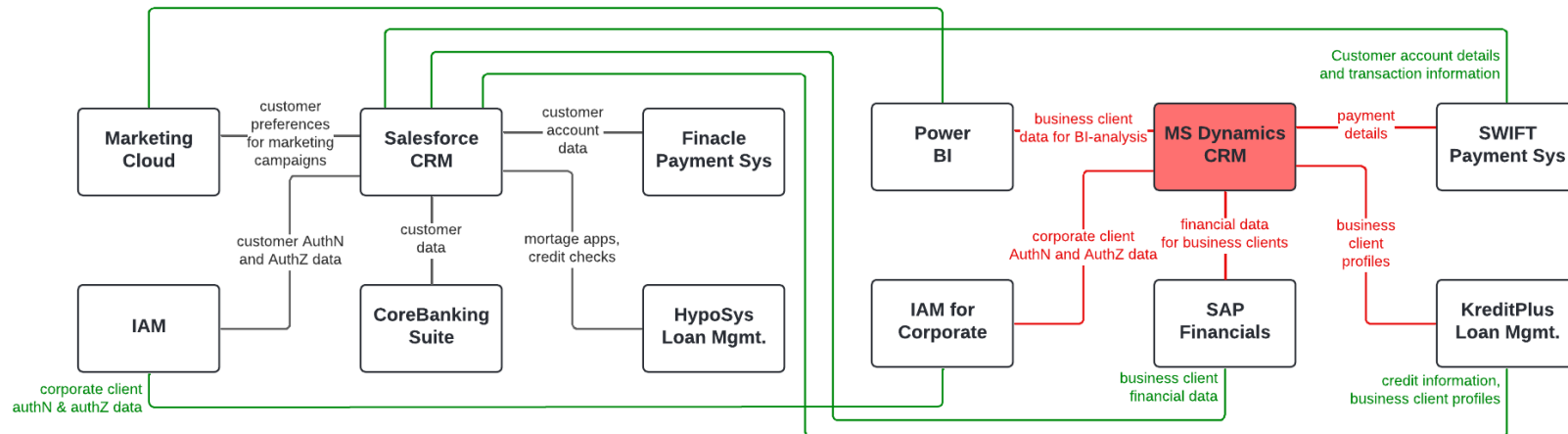
Enterprise, Domain, and Solution Architects reflect changes to the existing enterprise landscape resulting from the new design in the architecture repository.



Adjusted Building Blocks include, for example, **Business Processes, Capabilities or Organization** (Business Architecture ...)

... **Applications, Information Objects and Interfaces** (IT System Architecture) ...

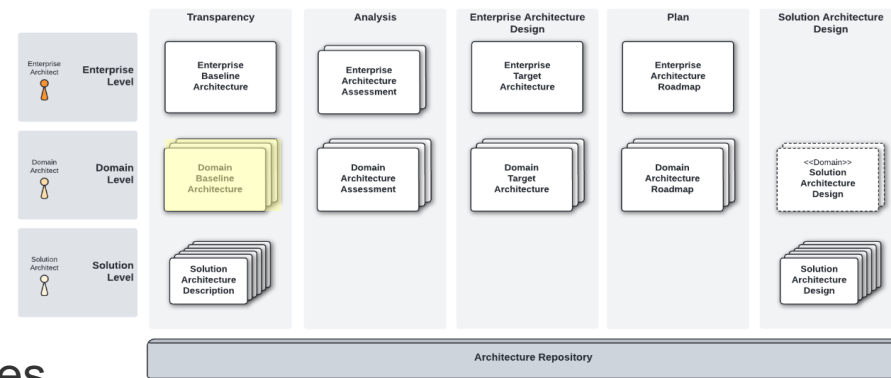
... and **Infrastructure Components** (Technology Architecture).



Methodology

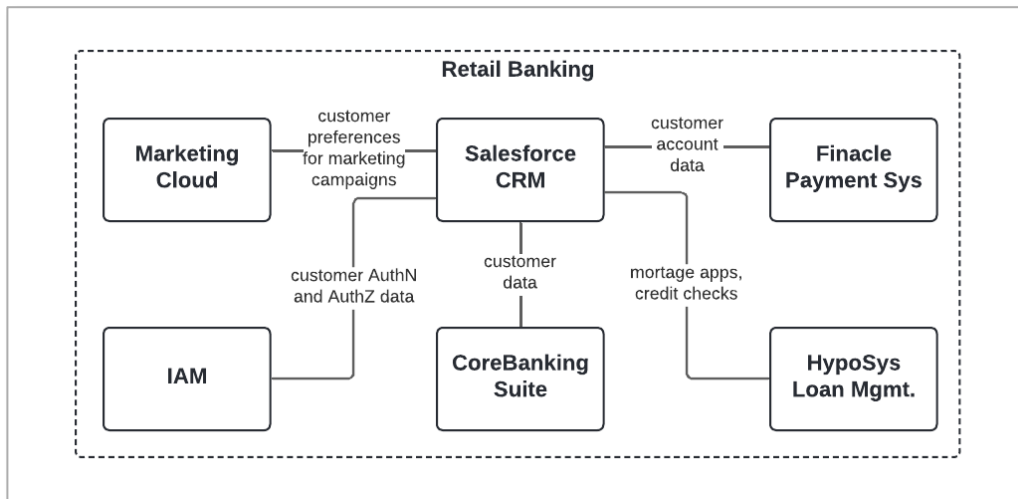
End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Domain Architects further reflect **landscape changes** on **artifacts** at the **Domain Architecture** level.

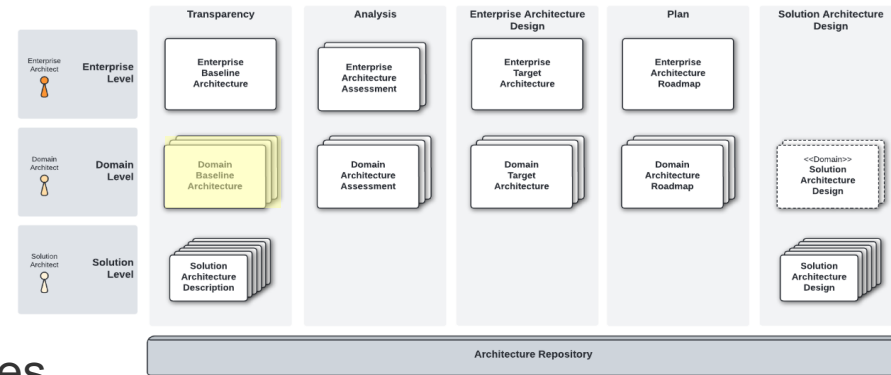
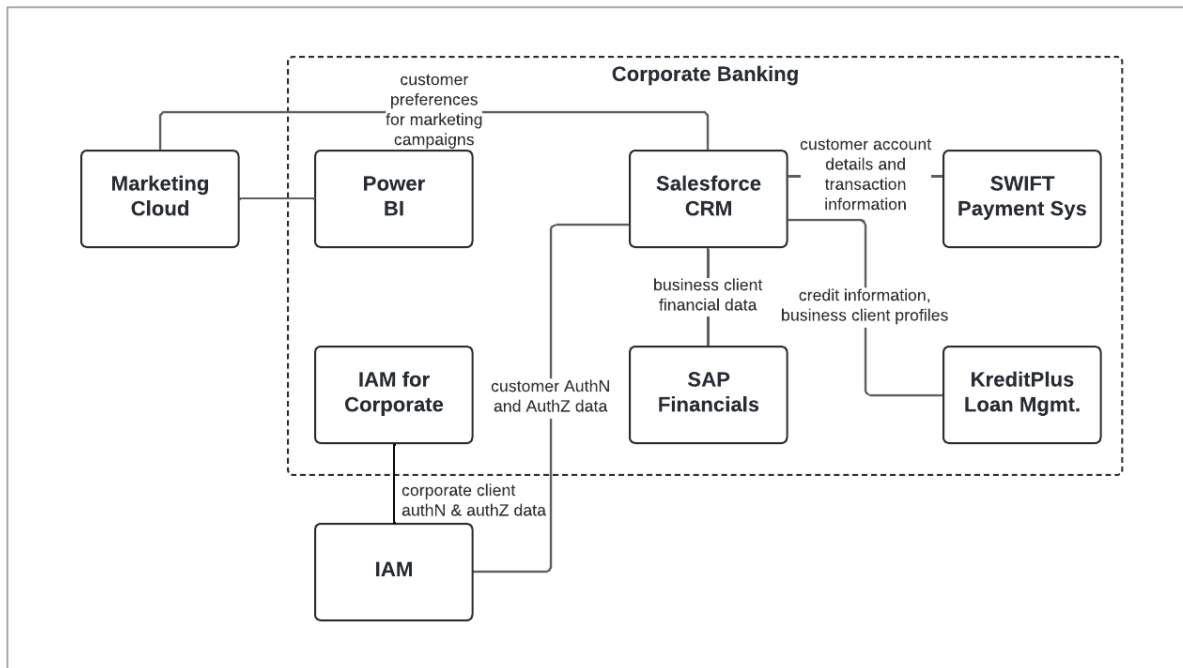
However, the **Retail Banking Architect** does **not** need to make any changes to his **Domain Baseline Architecture**, as it is **not** affected by the recent landscape change.



Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.

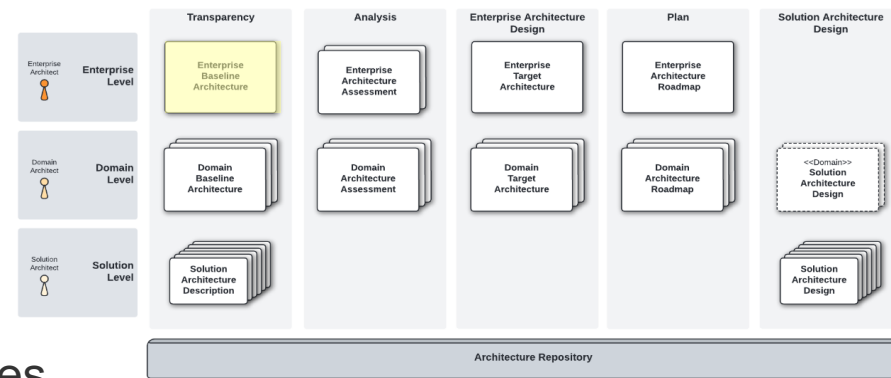


The **Corporate Banking Domain Architect**, on the other hand, reflects the design change by making the current Domain Target Architecture the new Domain Baseline Architecture.

Methodology

End-2-End Example

Using a **holistic example** that encompasses all **three levels**, we want to **show** which **artefacts** are created and **how they interlink**.



Finally, the **Enterprise Architect** sees an appropriately updated **Enterprise Baseline Architecture** based on the recently changed **Building Blocks** in the **repository**.

The **overview** shows at a **glance** that the **apps redundancy** regarding the **CRM Capability** has been resolved across both domains ...

... and that there is hence **no further need** for **consolidation** in this field.

Capability	Customer Relationship Management	Loan Management	Financial Planning	Payment Processing	Data Analysis	Identity & Access Mgmt.
Domain						
Retail Banking	Salesforce CRM	HyperSys Loan Mgmt.	CoreBanking Suite	Finacle Payment Sys	Marketing Cloud	IAM
Corporate Banking		KreditPlus Loan Mgmt.	SAP Financials	SWIFT Payment Sys	Power BI	IAM for Corporate

Lecture Agenda



- Methodology
- Architecture Mandate
- Solution Architecture Methodology

Architecture Mandate

Overview

The **architecture discipline collaborates** with other disciplines along the enterprise value chain.

Some collaborations are lightweight, informal by nature and part of normal day-to-day operations. Other collaborative relationships, however, require clarification of the expectations, assumptions, and commitments of all cooperating parties.

Inadequately aligned cooperation inevitably leads to ambiguity about scope, objective, deliverables, timelines, or resources.

The purpose of an **architecture mandate is to encourage and require appropriate clarification** before any commitments are made. A well-defined mandate promotes clarity of expectations on all sides by fully describing, and formally agreeing on commitments.

Architecture Mandate

Overview

Why are architecture mandates so crucial?

More often than we would like to admit, **we act without having sufficiently clarified the goal of our actions** for ourselves.

The reason that this happens to us is that we are all too **often hastily satisfied with an initial understanding of a given problem**.

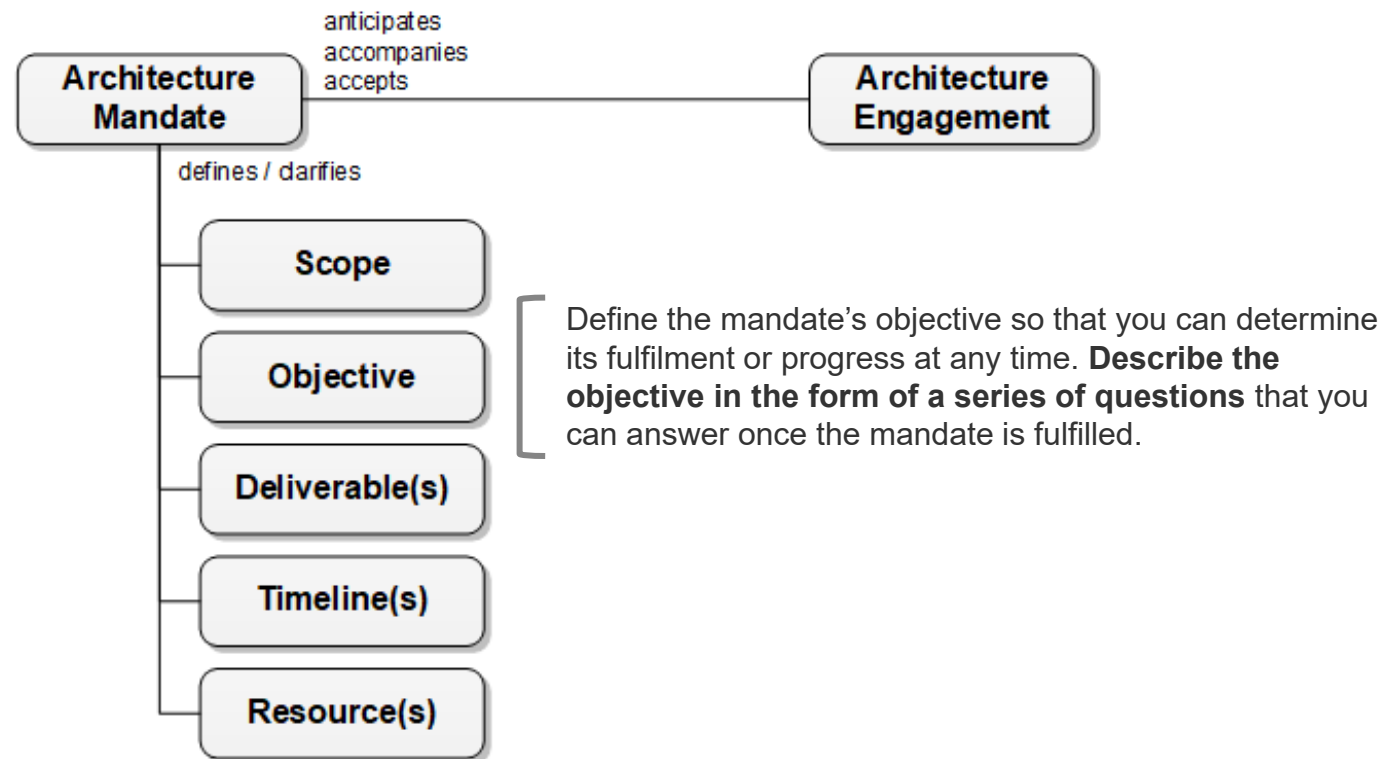
We **do not listen enough** or **reflect long enough** to distinguish between a *supposed understanding* of the problem (immature understanding) and an actual understanding of a given problem (mature understanding).

Architecture **mandates increase our awareness** as they force us to go through a dedicated reflection process.

Architecture Mandate

Architecture Mandate Specification

Architecture mandates are used to precisely **anticipate and articulate the desired end state of an architecture activity** and guide the path to its achievement



Architecture Mandate

Architecture Mandate Specification

Once defined, architecture mandates **support architects in performing the mandated engagements**.

While performing architecture activities in a corresponding engagement, architects use mandates to check whether their contributions have already achieved the desired goal (i.e., mandates serve architects as a *definition of done*).

Mandates, therefore, **accompany architecture engagements as a kind of yardstick**. This yardstick is continuously used to specifically plan and implement architecture contributions and to determine when they have been sufficiently delivered. Therefore, mandates are often incorporated in methods as a calibration companion.

Lecture Agenda



- Methodology
- Architecture Mandate
- Solution Architecture Methodology

Solution Architecture Methodology

Overview

Solution architecture methodologies standardize both the development and description of asset architectures.

That is, each solution methodology encompasses both a **development process** and a **view model**.

While a solution architecture development process defines the activities to be performed, a view model specifies what information is captured for which viewpoints to create an architecture description.

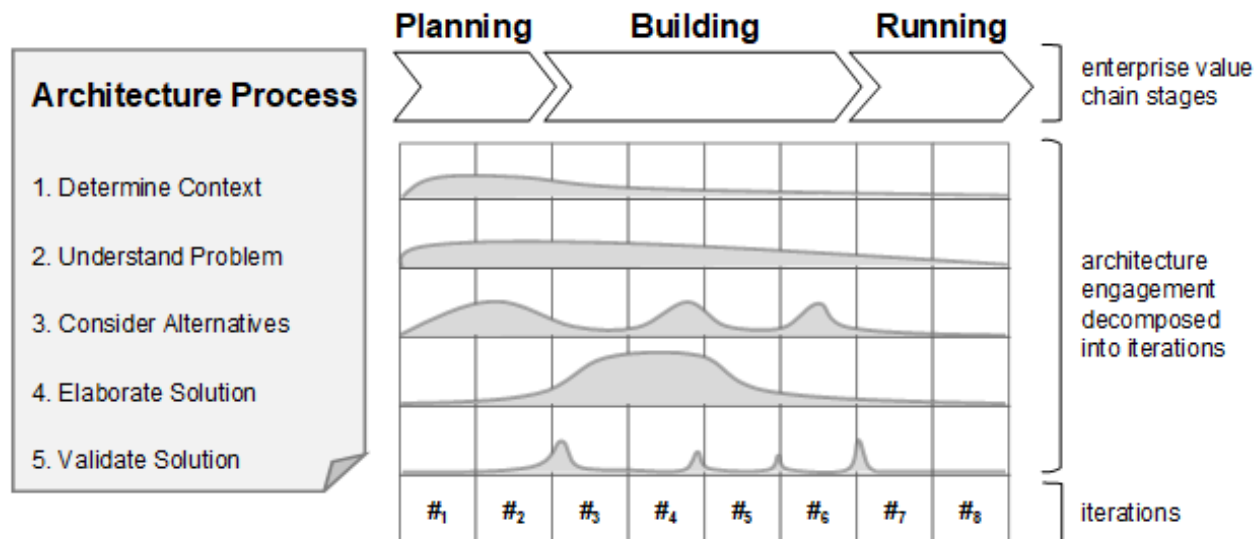
A well-designed solution methodology takes a **holistic approach to developing, capturing, and describing the solution design** in question. It avoids monocausal thinking and monodimensional capture of the architecture but strives to absorb the relevant dimensions of the given problem.

Furthermore, solution methodologies must consider that both **problems and their solutions evolve and change over time**. Thus, any meaningful methodology must address the evolutionary nature of problem-solving.

Solution Architecture Methodology

Overview

A **solution description is developed incrementally** through the repeated application of what appears to be a sequential process. Accordingly, one view of a solution view model is not completed at a time. Rather, all relevant views are also repeated again and again – and thus established incrementally.



Solution Architecture Methodology

Overview

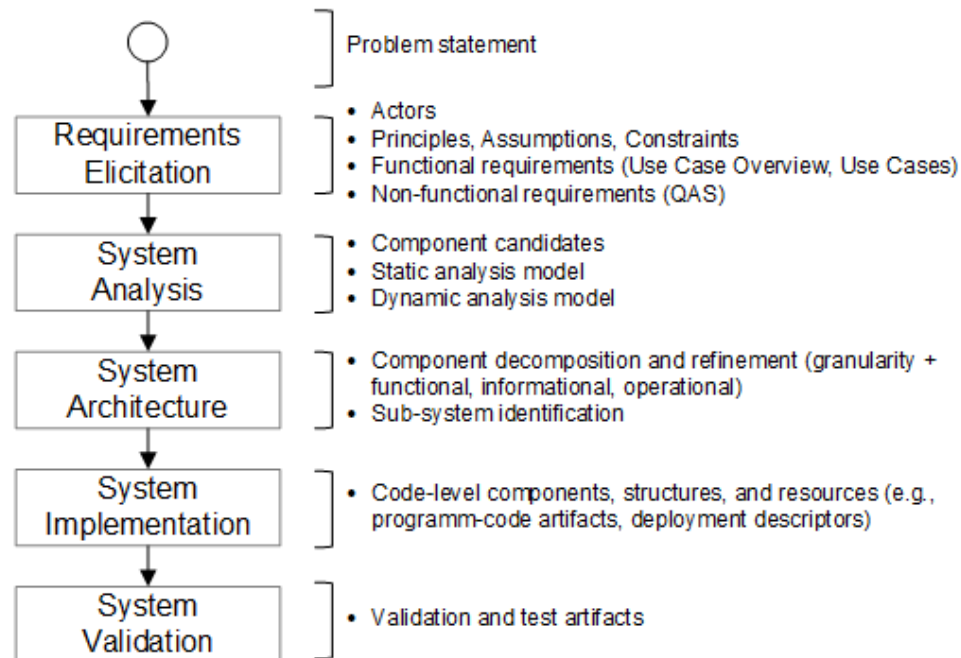
The procedure outlined below is certainly not sufficient in detail and scope, but it contains the genuinely essential ingredients of a solution architecture elaboration process.

- **Determine architecture context.** Determine the enterprise context of a given problem and envisioned solution. Understand how the enterprise context affects an envisioned solution (inbound dependencies) and, conversely, how a solution potentially affects its context (outbound dependencies).
- **Determine and refine architecture conditions.** Within the given conditions, identify those that are architecture-significant. Architecture conditions will significantly shape the design of your target architecture
- **Develop architecture alternatives and select alternative.** Investigate and develop alternative approaches to solve a given problem. Conduct this activity based on problem-specific selection metrics and select the most appropriate alternative based on them. In later iterations, repeat this activity in recursive descent for individual solution building blocks of the overall architecture.
- **Elaborate solution architecture.** Develop a solution design to respond to the identified architecture conditions and ensure that a given mandate is accomplished. Establish traceability across coherent views in such a way that you create a holistically navigable design description.
- **Validate solution architecture.** Perform architecture validation at the end of major progress, such as at the end of each iteration. What you specifically validate depends on where you are in your solution development.

Solution Architecture Methodology

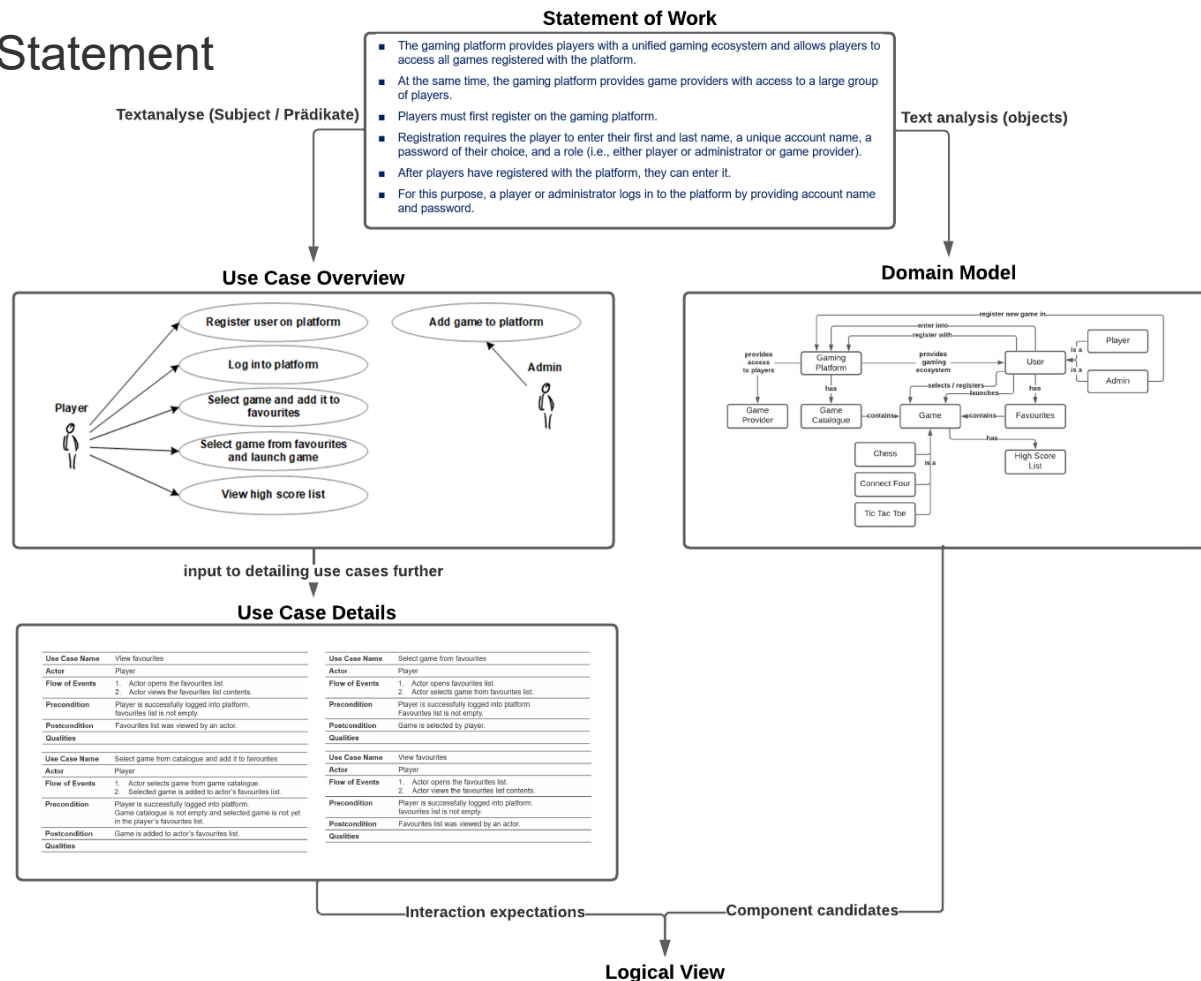
Problem-solving core

The core of the process spanning both, requirements and solutioning activities in is outlined in more detail, below.



Solution Architecture Methodology From Problem to Solution

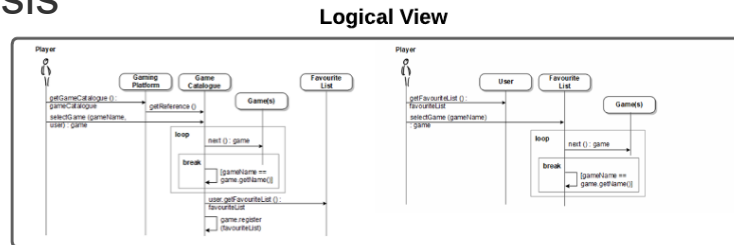
Problem Statement



Solution Architecture Methodology

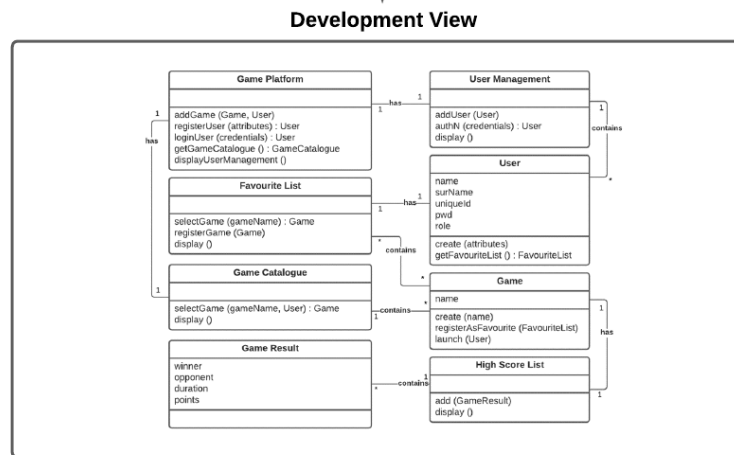
From Problem to Solution

System Analysis



In use case realisations, functional components and their methods are identified. The set of all these components is able to realise all functional expectations.

Generalising the components in classes and relationships between classes



If all essential use cases have been fully considered, the essential functional (business) components as well as their functional shells have been identified through this approach. Furthermore, it can be assumed that these components and their functional shells will not change significantly.

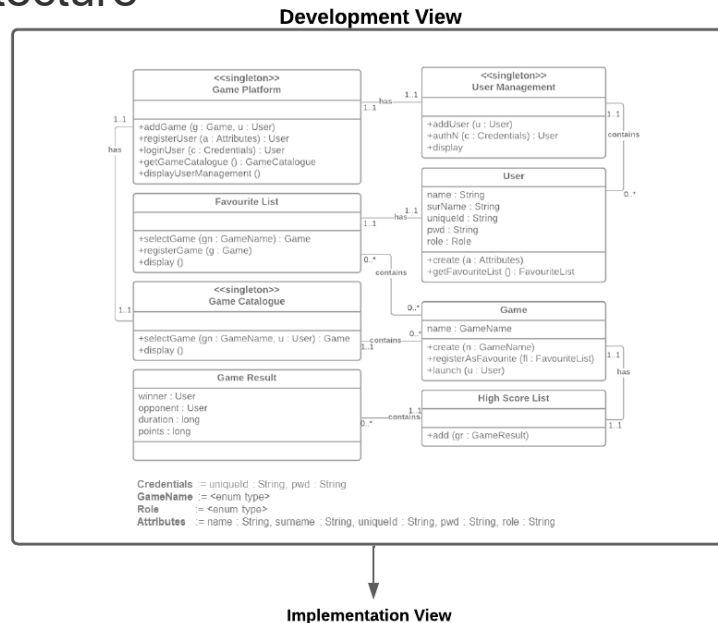
Elaborating System Architecture

Development View

Solution Architecture Methodology

From Problem to Solution

System Architecture



At the system architecture level, the analysis model is enriched with design-related details such as patterns, data types, further components.

Solution Architecture Methodology

From Problem to Solution

System Implementation

Implementation View

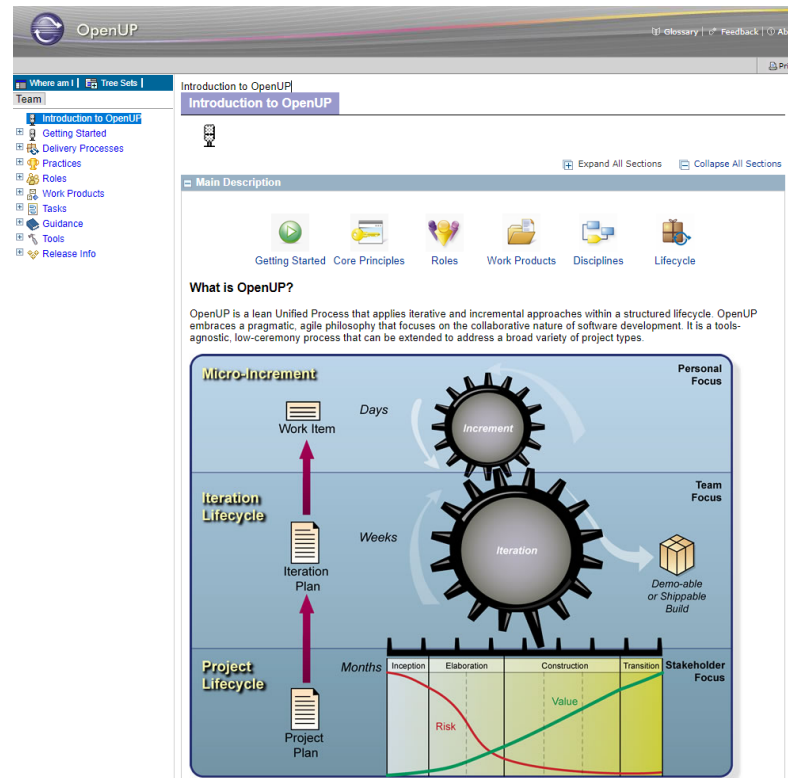
```
public class GamePlatform {  
    private static GamePlatform singleton = null;  
  
    // singleton factory  
    private GamePlatform() {}  
    public static GamePlatform getGamePlatform() {  
        if (singleton == null) {  
            singleton = new GamePlatform();  
            return singleton;  
        }  
    }  
  
    // instance methods  
    public void registerUser(String name, String surname, String uniqueId, String pwd1, String pwd2, Role role) {  
        UserManagement um = UserManagement.getUserManagement();  
  
        if (pwd1.contentEquals(pwd2)) {  
            User u = new MyUser(name, surname, uniqueId, pwd1, role);  
            um.addUser(u);  
        }  
    }  
  
    public void addGame(Game game, User user) {  
        // is user authenticated?  
        if (user != null) {  
            // does user have admin role?  
            if (user.getRole().equals(Role.ADMIN)) {  
                GameCatalogue gc = GameCatalogue.getGameCatalogue();  
                gc.add(game);  
            }  
        }  
    }  
}
```

On the basis of the models at system architecture level, a solution is implemented with suitable implementation language/technology. In the code artefacts "model" at implementation level, the software developer has degrees of freedom in design that are not predetermined by the system architecture.

Solution Architecture Methodology

Problem-solving framework

OpenUP [OpenUP 2021] is a process instance of the eclipse process framework (EPF) – an eclipse project.



Wrap-Up

■ Methodology

- **Definition:** A methodology systematically outlines activities toward objectives, defines states and transitions, and specifies roles, inputs, and outputs.
- **Purpose:** increases systematicity, repeatability, alignment, and governance; fosters reuse instead of reinvention.
- **Types:**
 - **Solution Architecture Development** (e.g., Unified Process, TOGAF, ArchiMate)
 - **Enterprise/Domain Architecture Development** (e.g., TOGAF, Zachman)
 - **Reference Architecture Development** (patterns, roadmaps)
 - **Architecture Assessment** (e.g., ATAM)
- **Misconceptions:** Frameworks are tools, not goals; they must be adapted to context and objectives.

■ Framework Adoption

- **Frameworks** (e.g., TOGAF ADM) are **generic** and **must be tailored** to **business context** and **objectives**.
- **Adaptation requires**
 - **Clarifying** the **value proposition** of architecture.
 - **Identifying stakeholders, objectives, benefits** (quantified), and **costs**.
 - **Adjusting framework elements** at **different levels** (enterprise, domain, solution).

Wrap-Up

■ Architecture Process

- **Structured, iterative methodology embedded in frameworks** (e.g., TOGAF ADM).
- **Key phases:**
 - **Transparency & Analysis:** baseline, gaps, redundancies, risks, opportunities.
 - **Design:** business architecture → IT landscape (apps, data, tech).
 - **Planning:** target architecture roadmap, portfolio management.
 - **Governance:** guardrails (business, organizational, regulatory, technical).
 - **Requirements Management:** central across all phases.
- **Iterative cycles:** context setup, delivery (baseline → target), transition planning, governance.
- **Processes apply across Enterprise, Domain, Solution Architecture; linked to view models** as representation artifacts.
- **End-to-end methodology** spans **transparency, analysis, planning**, and **reflection** into **architecture repositories**.

Wrap-Up

■ Architecture Mandate

- **Definition:** A **mandate** is a **formal clarification** of **expectations, assumptions, objectives**, and **commitments** for an **architecture engagement**.
- **Purpose:** **prevents ambiguity, promotes shared understanding, defines objectives** as well as “**definition of done**”.
- **Role:** **serves** as a **yardstick** during **engagements, guiding scope, deliverables**, and **when work is sufficiently completed**.

Wrap-Up

■ Solution Architecture Methodology

- **Definition:** Standardizes the **development** and **description** of **solution architectures**.
- **Structure**
 - **Development process** (activities performed).
 - **View model** (information captured per viewpoint).
- **Characteristics**
 - **Holistic, multi-dimensional** (avoids monocausal solutions).
 - **Evolutionary and iterative** (solutions evolve with context).
 - **Incremental development** of all **views** in **parallel**.
- **Essential steps**
 1. **Determine architecture context** (inbound/outbound dependencies).
 2. **Identify and refine architecture conditions** (focus on significant ones).
 3. **Develop alternatives**, evaluate against metrics, and select best option.
 4. **Elaborate solution architecture** (traceable, holistic views).
 5. **Validate solution architecture** (per iteration).
- **Problem-to-solution path:** Problem Statement → System Analysis → System Design → System Implementation.
- **Example methodology:** OpenUP, a lightweight iterative framework.

Questions



Bibliography

Lecture

[Arnold 2021]

Arnold, Ingo, *Enterprise Architecture Function — a pattern language for planning, designing, and executing*, Springer Science and Business Media, Berlin Heidelberg, 2021

[TOGAF® 2021]

Open Group, *The Open Group Architecture Framework*, 2021,
<https://www.opengroup.org/togaf>

[OpenUP 2021]

Eclipse, Eclipse process framework – OpenUP, 2021,
<https://www.utm.mx/~caff/doc/OpenUPWeb/>